

Dedication

To my dear parents,

*for their unconditional love, sacrifices, prayers, and
constant support.*

To my family,

*for their encouragement, patience, and confidence
throughout this journey.*

To my teachers and mentors,

for their guidance, inspiration, and valuable advice.

I dedicate this work with deep gratitude and respect.

AIT-LAHCEN REDOUAN

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor for the guidance, availability, constructive feedback, and encouragement provided throughout this final year project. The advice received helped me refine the scope of the work, strengthen the technical choices, and maintain a disciplined engineering approach from design to implementation.

I would also like to thank the academic staff whose teaching and methodological support shaped the foundations of this project. The knowledge acquired during my Master's program directly contributed to my understanding of data engineering, distributed systems, databases, software development, and the design of reliable information platforms.

My appreciation also goes to the professional environment that made this work possible and gave it practical relevance. Developing this project within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES allowed me to understand real data engineering constraints and to position this work as a distributed customer data platform intended to support trustworthy analytics, structured governance, and future AI-oriented developments.

I am equally grateful to my colleagues and classmates for their discussions, exchanges, and mutual support during this academic journey. Their presence made the periods of experimentation, debugging, and refinement both more productive and more meaningful.

I extend my deepest thanks to my family for their patience, encouragement, and constant confidence throughout the long phases of research, implementation, testing, and writing. Their support remained an essential source of motivation from the beginning of the project to its completion.

Finally, I would like to thank the members of the jury for the time and attention devoted to evaluating this work. I hope that this report faithfully reflects the seriousness, effort, and commitment invested in the design of a modern, distributed, and production-oriented data engineering platform.

Abstract

Modern organizations produce large volumes of customer-related data through heterogeneous and fragmented operational environments. Campaign records, behavioral traces, commercial transactions, and profile attributes are often generated in different formats, at different rhythms, and under different semantic assumptions. When these sources remain isolated, the organization faces a structural difficulty: customer information becomes hard to trust, hard to reconcile, hard to replay, and hard to transform into stable analytical assets. Decision-making is therefore slowed down, cross-source understanding remains partial, and future intelligent services lack a sufficiently reliable data foundation.

This final year project addresses that problematic through the design and implementation of AdOptimizer CDP, a distributed customer data lakehouse platform developed within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The objective is not to deliver a final artificial intelligence product, but to establish the engineering foundation required to make future analytics and AI-oriented use cases technically credible. The work therefore focuses on structuring the complete lifecycle of customer-related data, from multi-source ingestion and durable raw persistence to distributed processing, curated analytical organization, quality control, orchestration, observability, and controlled access.

The contribution of the project lies in building a coherent end-to-end platform able to convert heterogeneous raw datasets into governed and reusable data products within a distributed environment. The implemented architecture is designed to improve reproducibility, scalability, traceability, and operational reliability while preserving the flexibility required by evolving customer-data use cases. Beyond the technical implementation, the project demonstrates how a modern data engineering platform can serve as the essential foundation for trusted analytics, future Customer 360 capabilities, and subsequent machine learning developments.

Keywords: Big Data, Data Engineering, Distributed Systems, Data Lakehouse, Customer Data Platform, Data Governance, Data Quality, Observability, Security, Customer 360.

Résumé

Les organisations modernes produisent des volumes importants de données client dans des environnements opérationnels hétérogènes et fragmentés. Les données de campagne, les traces comportementales, les transactions commerciales et les attributs de profil sont souvent générés dans des formats différents, selon des rythmes distincts et avec des hypothèses sémantiques non homogènes. Lorsque ces sources restent isolées, une difficulté structurelle apparaît : l'information devient difficile à fiabiliser, à réconcilier, à rejouer et à transformer en actifs analytiques cohérents. Les processus de décision se ralentissent, la vision client demeure partielle et les futurs services intelligents ne disposent pas d'un socle de données suffisamment fiable.

Ce projet de fin d'études répond à cette problématique par la conception et la mise en oeuvre de AdOptimizer CDP, une plateforme distribuée de type lakehouse pour les données client, développée dans le cadre de l'initiative plus large AdOptimizer AI au sein de SMART AUTOMATION TECHNOLOGIES. L'objectif n'est pas de livrer un produit d'intelligence artificielle finalisé, mais d'établir la fondation d'ingénierie nécessaire pour rendre crédibles de futurs cas d'usage analytiques et orientés IA. Le travail porte ainsi sur l'organisation du cycle de vie complet des données client, depuis l'ingestion multi-source et la persistance durable des données brutes jusqu'au traitement distribué, à la structuration analytique gouvernée, au contrôle qualité, à l'orchestration, à l'observabilité et à l'accès maîtrisé.

La contribution principale du projet consiste à construire une plateforme cohérente de bout en bout capable de transformer des jeux de données hétérogènes en produits de données gouvernés et réutilisables dans un environnement distribué. L'architecture mise en oeuvre vise à renforcer la reproductibilité, la scalabilité, la traçabilité et la fiabilité opérationnelle, tout en conservant la flexibilité nécessaire à l'évolution de futurs usages centrés sur la donnée client. Au-delà de l'implémentation technique, ce projet montre comment une plateforme moderne d'ingénierie de données peut constituer le socle indispensable d'analyses fiables, de futures capacités Customer 360 et de développements ultérieurs en machine learning.

Mots-clés: Big Data, Ingénierie des données, Systèmes distribués, Data Lakehouse, Plateforme de données client, Gouvernance des données, Qualité des données, Observabilité, Sécurité, Customer 360.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
List of Acronyms and Abbreviations	xii
I General Project Context	1
I.1 Introduction	1
I.2 Host Organization and Project Environment	1
I.2.1 Company and Strategic Context	1
I.2.2 Company Values and Their Relevance to the Project	2
I.2.3 Position of the Project Within AdOptimizer AI	3
I.3 Target Project Presentation	3
I.3.1 Problem Statement	3
I.3.2 Motivation for a Distributed Lakehouse Approach	4
I.3.3 Platform Identity and Academic Framing	4
I.3.4 Main Objectives of the Project	5
I.3.5 Implemented Scope and Project Boundaries	5
I.3.6 Expected Results and Academic Contribution	6
I.4 Conclusion	6
II State of the Art and Theoretical Background	7
II.1 Introduction	7
II.2 Foundations of Modern Distributed Data Systems	7
II.2.1 From Centralized Systems to Distributed Data Platforms	7
II.2.2 Vertical Scaling vs Horizontal Scaling	10
II.2.3 Reliability, Scalability, and Maintainability	11
II.2.4 Data-Intensive Systems as a Distinct Engineering Problem	11
II.3 Core Theoretical Trade-offs in Distributed Data Engineering	11

II.3.1	The CAP Theorem and Consistency Trade-offs	11
II.3.2	ACID vs BASE Models	12
II.3.3	Data Partitioning and Replication	12
II.3.4	Fault Tolerance and Service Resilience	13
II.3.5	Batch Processing vs Stream Processing	13
II.3.6	Schema-on-Write vs Schema-on-Read	13
II.4	Big Data Foundations	14
II.4.1	The Five Vs of Big Data	14
II.4.2	Heterogeneous Data Sources and Data Variety	14
II.4.3	Distributed Storage Principles	14
II.4.4	Distributed Compute Principles	15
II.5	Evolution of Analytical Data Architectures	16
II.5.1	Traditional Data Warehouse Architecture	16
II.5.2	Data Lake Architecture	16
II.5.3	Data Lakehouse Architecture	16
II.5.4	Lambda Architecture	16
II.5.5	Kappa Architecture	17
II.5.6	Comparative Analysis of These Architectures	17
II.6	Foundational Concepts for Governance and Trust in Data Platforms . .	18
II.6.1	Metadata, Cataloging, and Data Discovery	18
II.6.2	Data Lineage and Traceability	18
II.6.3	Data Quality Dimensions	19
II.6.4	Observability in Distributed Data Systems	19
II.7	Security Foundations for Distributed Data Platforms	20
II.7.1	Authentication in Distributed Systems	20
II.7.2	Authorization and Access Control	20
II.7.3	Encryption in Transit and Encryption at Rest	20
II.7.4	Secure Service Exposure in Multi-Node Environments	20
II.8	Conclusion	21
III	Technology Watch	22
III.1	Introduction	22
III.2	Selection Criteria	22
III.3	Apache Kafka	23
III.4	Hadoop HDFS	23
III.5	Apache Iceberg and Hive Metastore	23
III.6	Apache Spark	24
III.7	Trino	24
III.8	dbt for Transformation Modeling	25

III.9	Great Expectations	25
III.10	Apache Airflow	26
III.11	Prometheus and Grafana	26
III.12	Docker	27
III.13	Synthesis of the Final Stack	27
III.14	Conclusion	28
IV	Requirements Analysis and System Design	29
IV.1	Introduction	29
IV.2	Requirement Elicitation Context	29
IV.3	Functional Requirements	30
IV.4	Non-Functional Requirements	31
IV.5	Global Logical Architecture	31
IV.6	Physical Multi-Node Architecture	33
IV.7	End-to-End Data Flow	34
IV.8	Security-by-Design Integration	35
IV.9	Design Rationale and Trade-offs	36
IV.10	Conclusion	36
V	Implementation of the Distributed Lakehouse Platform	38
V.1	Introduction	38
V.2	Data Sources and Dataset Modeling	38
V.3	Kafka-Based Ingestion Layer	39
V.4	HDFS Bronze Storage Layer	40
V.5	Spark Processing and Iceberg Loading	41
V.6	Hive Metastore and Catalog Governance	42
V.7	Trino Query Layer	43
V.8	dbt-spark Transformation Layer	44
V.9	Great Expectations Validation Layer	45
V.10	Airflow Orchestration Layer	46
V.11	Conclusion	47
VI	Deployment Strategy and Distributed Execution	48
VI.1	Introduction	48
VI.2	Control Plane Responsibilities	48
VI.3	VM-Level Service Separation	49
VI.4	Data Flow Across Machines	50
VI.5	Distributed Processing Proof	52
VI.6	Operational Packaging and Deployment Bundles	53
VI.7	Conclusion	55

VII	Security Architecture and Hardening	56
VII.1	Introduction	56
VII.2	Security Objectives	56
VII.3	Kerberos-Secured Hadoop Services	57
VII.4	SPNEGO-Protected HDFS Browser Access	58
VII.5	TLS-Protected Interfaces	59
VII.6	Authenticated Trino Access	60
VII.7	SSH-Based Remote Spark Submission	61
VII.8	Encrypted Runtime Storage Support	62
VII.9	Security Boundaries and Remaining Limits	63
VII.10	Conclusion	64
VIII	Validation, Testing, and Results	65
VIII.1	Introduction	65
VIII.2	Validation Strategy	65
VIII.3	Infrastructure Validation	66
VIII.4	Pipeline Validation	67
VIII.5	Distributed Execution Validation	68
VIII.6	Security Validation	69
VIII.7	Observability and Monitoring Validation	69
VIII.8	Main Results	70
VIII.9	Conclusion	71
IX	Discussion and Critical Analysis	72
IX.1	Introduction	72
IX.2	Strengths of the Final Architecture	72
IX.3	Trade-Offs and Constraints	73
IX.4	Academic Value	74
IX.5	Industrial Value	74
IX.6	Critical Limits	75
IX.7	Conclusion	75
X	General Conclusion and Perspectives	77
X.1	General Conclusion	77
X.2	Main Achievements	77
X.3	Future Perspectives	78
A	Appendices	81

List of Figures

I.1	SMART AUTOMATION TECHNOLOGIES visual identity	2
I.2	Core values of SMART AUTOMATION TECHNOLOGIES	2
II.1	Centralized architecture as a concentration of ingestion, storage, processing, and access on one execution point.	8
II.2	Distributed data platform as a layered decomposition of storage, compute, metadata, and analytical access responsibilities.	9
II.3	Conceptual view of distributed execution in a Spark-style compute model.	15
II.4	Conceptual positioning of the implemented project between replay-oriented ingestion logic and batch-structured analytical refinement. . .	18
III.1	Technology retained for event ingestion and topic-based transport . . .	23
III.2	Technology retained for distributed bronze-layer storage	23
III.3	Technologies retained for governed lakehouse tables and metadata management	24
III.4	Technology retained for distributed processing and analytical loading .	24
III.5	Technology retained for interactive SQL access	24
III.6	Technologies retained for model-driven transformations and analytical storage format support	25
III.7	Technology retained for explicit data quality validation	25
III.8	Technology retained for workflow orchestration	26
III.9	Technologies retained for observability and dashboarding	26
III.10	Technology retained for service isolation and reproducible deployment .	27
IV.1	Global logical architecture of the platform from ingestion to analytical access, validation, and orchestration.	32
IV.2	Physical multi-node architecture showing the local control plane and the service placement across VM1, VM2, and VM3.	34
IV.3	End-to-end flow from heterogeneous source datasets to validated analytical outputs.	35
V.1	Active dataset families and their complementary contribution to the customer-oriented analytical scope of the platform.	39

V.2	Dataset publication flow from the control plane to the three-broker Kafka cluster and toward bronze persistence.	40
V.3	Bronze storage organization in HDFS, with NameNode coordination on VM2 and DataNode distribution across VM1, VM2, and VM3.	41
V.4	Distributed Spark raw-load path from replayable bronze batches to governed raw lakehouse tables.	42
V.5	Shared metadata governance through Hive Metastore between distributed writers and analytical readers.	43
V.6	Protected Trino analytical access path from secure entry point to coordinator and distributed workers.	44
V.7	Layered dbt transformation flow from raw tables to staging, intermediate, and analytical outputs.	45
V.8	Validation checkpoints over the lakehouse layers and the persistence of validation evidence.	46
V.9	Five-DAG orchestration sequence from environment preparation to raw loading and curated transformation.	47
VI.1	Separation between the local control plane and the distributed Linux data plane.	49
VI.2	Service placement map across VM1, VM2, and VM3.	50
VI.3	Machine-level data flow from local source handling to distributed analytical exposure.	52
VI.4	Spark master and worker collaboration across VM1, VM2, and VM3 during distributed execution.	53
VI.5	Deployment lifecycle from bundle preparation to preflight, startup order, and control-plane activation.	54
VII.1	Kerberos-centered identity flow between the KDC and the protected Hadoop service layer.	58
VII.2	Secure HDFS browser path from Windows Kerberos ticket to SPNEGO-authenticated HTTPS endpoint.	59
VII.3	TLS-protected service exposure zones for HDFS, Trino, and the local control plane interfaces.	60
VII.4	Authenticated Trino gateway in front of coordinator and worker execution.	61
VII.5	Secured control-plane-to-VM2 remote Spark submission path over SSH.	62
VII.6	Security coverage map distinguishing identity, browser access, traffic encryption, and encrypted runtime storage.	63
VII.7	Implemented security boundary versus out-of-scope future enterprise hardening controls.	64

List of Tables

II.1	Comparison between centralized and distributed architectural logic . .	10
IV.1	Primary functional requirements of the platform, part 1	30
IV.2	Primary functional requirements of the platform, part 2	30
IV.3	Main non-functional requirements of the platform	31
IV.4	Physical node distribution of the final deployment	33
IV.5	Security controls integrated into the system design	36
V.1	Active datasets retained in the implemented platform	39
V.2	Implemented orchestration flow of the platform	46
VI.1	Main responsibilities of the local control plane	49
VI.2	VM-level service allocation in the distributed deployment	49
VI.3	Operational deployment artifacts used in the project	54
VII.1	Main security objectives and corresponding implementation controls . .	57
VIII.1	Validation strategy adopted for the deployed platform	66
VIII.2	Operational meaning of the core DAG validations	68
IX.1	Main constraints and the corresponding design responses	73

List of Acronyms and Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CDP	Customer Data Platform
DAG	Directed Acyclic Graph
dbt	Data Build Tool
ETL	Extract, Transform, Load
GE	Great Expectations
HDFS	Hadoop Distributed File System
JDBC	Java Database Connectivity
JSONL	JSON Lines
KDC	Key Distribution Center
KPI	Key Performance Indicator
LTV	Lifetime Value
ML	Machine Learning
PFE	Projet de Fin d'Etudes
RBAC	Role-Based Access Control
SQL	Structured Query Language
SPNEGO	Simple and Protected GSSAPI Negotiation Mechanism
TLS	Transport Layer Security
UI	User Interface
VM	Virtual Machine

Chapter I

General Project Context

I.1 Introduction

This chapter introduces the academic, business, and engineering context of AdOptimizer CDP. Its objective is to explain the environment in which the project was conducted, the broader strategic initiative that motivates it, the business problem it addresses, and the exact positioning of the present final year project within that larger transformation effort. It also clarifies the institutional origin of the work: the project was proposed and framed by SMART AUTOMATION TECHNOLOGIES as part of the professional context in which the present PFE was carried out.

I.2 Host Organization and Project Environment

I.2.1 Company and Strategic Context

AdOptimizer CDP was developed in the context of SMART AUTOMATION TECHNOLOGIES, the company that assigned and supported the project as the professional framework of this final year work. SMART AUTOMATION TECHNOLOGIES is a technology-oriented environment whose positioning is centered on digital innovation, automation, and intelligent systems. In the present project, the company context is important because it provides the industrial logic behind the need for a robust customer-data engineering platform. Rather than approaching data as a secondary reporting artifact, the project treats data as a strategic technical asset that must be ingested, governed, processed, exposed, validated, monitored, and secured in a coherent way.

The broader strategic direction in which this work is positioned is AdOptimizer AI. This global initiative represents the company's long-term vision for an intelligent advertising and



Figure I.1: SMART AUTOMATION TECHNOLOGIES visual identity

marketing ecosystem able to support advanced customer understanding, future analytics, and data-driven optimization. The present PFE project was therefore not chosen in isolation; it was given by SMART AUTOMATION TECHNOLOGIES within that broader strategic orientation. However, such a vision cannot be implemented reliably without a strong data foundation. For that reason, the PFE contribution focuses specifically on the distributed customer-data platform layer required to support future intelligent services.

I.2.2 Company Values and Their Relevance to the Project

The company's institutional values provide a meaningful lens for understanding the spirit of the project. The visual material associated with SMART AUTOMATION TECHNOLOGIES highlights five major values: **Innovation**, **Team Spirit**, **Integrity**, **Excellence**, and **Commitment**. These values are not merely symbolic; they align naturally with the logic of a modern data engineering platform.



Figure I.2: Core values of SMART AUTOMATION TECHNOLOGIES

Innovation is reflected in the adoption of a modern lakehouse architecture instead of a

conventional isolated reporting workflow. Team spirit is reflected in the platform’s role as a shared technical foundation that can later serve engineers, analysts, and future AI-oriented stakeholders. Integrity is reflected in the emphasis on trusted data, validation, reproducibility, and security. Excellence is reflected in the engineering objective of building a coherent end-to-end platform rather than assembling disconnected tools. Commitment is reflected in the effort to stabilize the platform operationally and make it defensible both technically and academically.

I.2.3 Position of the Project Within AdOptimizer AI

Within the broader AdOptimizer AI vision, the present PFE project must be positioned as AdOptimizer CDP, that is, the **distributed customer data lakehouse platform** that acts as the technical and engineering foundation of the larger initiative. This distinction is essential for the academic defense of the work.

The broader initiative is concerned with future intelligent marketing and advertising capabilities. Such capabilities may later include customer segmentation, conversion analysis, lifetime value analysis, churn-related reasoning, personalization, campaign optimization, and recommendation-oriented decision support. Nevertheless, these downstream intelligent functions are not the main implemented contribution of the present work. The actual contribution of the PFE is the design and implementation of the distributed and secure data platform that makes such future services possible and trustworthy.

This positioning is important because it prevents conceptual confusion during the defense. The project is not a finished machine learning product, nor is it a simple business intelligence dashboard. It is a Data Engineering and Big Data platform that delivers the foundational infrastructure necessary to sustain future Customer 360 and AI-driven use cases.

I.3 Target Project Presentation

I.3.1 Problem Statement

Customer-related data is naturally fragmented across different operational contexts. In modern organizations, such fragmentation can appear through campaign-response datasets, digital behavioral traces, session-level intent indicators, and commercial transaction histories. When these data sources remain isolated, they weaken analytical trust, complicate reprocessing, reduce reproducibility, and make it difficult to construct governed datasets for downstream consumers.

From an engineering perspective, the problem is therefore not only to store data, but to

create a structured platform able to ingest heterogeneous sources, preserve replayable raw artifacts, process them into governed lakehouse tables, expose them to analytical users, validate their quality, orchestrate execution reliably, and monitor operational behavior across a distributed environment. Without such a platform, future customer analytics remain fragile, difficult to scale, and hard to defend from a governance perspective.

The present project answers the following engineering question:

How can we build a distributed customer-data platform that ingests heterogeneous datasets through a reproducible pipeline, stores replayable raw data in distributed storage, processes it into governed lakehouse tables, exposes it for analytics, validates its quality, orchestrates it end to end, monitors the full system, and secures core infrastructure flows in a way that is defensible as a modern Big Data Engineering project?

I.3.2 Motivation for a Distributed Lakehouse Approach

The adoption of a distributed lakehouse architecture is motivated by both technical and academic reasons. From a technical point of view, the project needs to combine heterogeneous ingestion, replayable raw persistence, scalable processing, metadata governance, SQL exploration, transformation control, and data quality verification. From an academic point of view, the project must demonstrate that these capabilities are not isolated tasks, but parts of a coherent architectural response to a real-world data engineering problem.

The distributed lakehouse model is particularly appropriate because it allows the platform to keep raw data in distributed storage, process it with distributed compute resources, and expose curated analytical tables without forcing all data to be transformed immediately into a rigid warehouse-only model. It also supports separation of concerns between ingestion, storage, processing, transformation, validation, orchestration, and monitoring. This makes the architecture both more scalable and more defensible in front of a jury evaluating the project as a Data Engineering system rather than as a set of scripts.

I.3.3 Platform Identity and Academic Framing

The final implemented platform is best described as a **secure distributed customer-data engineering platform**. Its role is to establish the technical conditions under which customer-related data can be treated as a governed and reusable organizational asset. In this sense, the project integrates ingestion, distributed storage, distributed processing, lakehouse management, analytical querying, transformation governance, validation, orchestration, monitoring, and baseline infrastructure security into one operational foundation.

This positioning creates a strong academic identity for the report. The project is simultaneously:

- a Data Engineering platform,
- a Big Data architecture,
- a distributed lakehouse implementation,
- a secure multi-node data platform,
- and a deployment-oriented technical foundation for downstream Customer 360 and AI use cases.

I.3.4 Main Objectives of the Project

The project pursues a set of complementary objectives. First, it aims to organize heterogeneous customer-oriented datasets in a reproducible way. Second, it aims to transport these datasets through Kafka-based ingestion flows in order to create explicit and replayable movement patterns. Third, it aims to persist raw data in an HDFS bronze layer before converting it into structured analytical tables through Spark and Iceberg.

Beyond this core movement, the project also aims to centralize metadata through Hive Metastore, expose trusted analytical access through Trino, manage transformations through dbt-spark, validate data quality through Great Expectations, orchestrate the full sequence through Airflow, and monitor both the infrastructure and the pipelines through Prometheus, Grafana, and pipeline metrics. Finally, the project aims to demonstrate that these services can be deployed in a distributed and secured environment across multiple virtual machines rather than on a single isolated node.

I.3.5 Implemented Scope and Project Boundaries

The current implemented scope focuses on a customer-oriented multi-source bundle for Client 1. The platform integrates three active datasets that reflect complementary dimensions of customer understanding: campaign and profile data, online behavioral intent data, and commercial retail transaction data. This selection is sufficiently rich to justify future Customer 360 analytics while remaining compatible with the available infrastructure.

At the same time, it is important to state the project's boundaries clearly. The present work does not implement the final downstream machine learning modules such as dynamic segmentation, churn prediction, lifetime value prediction, persona generation, or recommendation engines. Instead, it implements the distributed and secure data foundation required

for such future capabilities. This distinction strengthens the rigor of the report because it prevents exaggerated claims and keeps the academic focus on the actual engineering contribution.

I.3.6 Expected Results and Academic Contribution

The implemented outcome of the project is a reproducible end-to-end data platform in which heterogeneous customer-related datasets are ingested, stored, processed, transformed, queried, validated, orchestrated, monitored, and secured. Concretely, the platform produces replayable bronze persistence in distributed storage, trusted analytical tables available through SQL, validation evidence usable in the final defense, and a distributed runtime architecture that demonstrates collaboration across multiple nodes.

From an academic perspective, the project contributes by showing how a modern customer-data platform can be designed as a coherent integration of event transport, distributed storage, distributed computation, metadata governance, analytical access, transformation control, quality assurance, orchestration, observability, and security hardening. The originality of the work does not lie in inventing new algorithms, but in engineering a defensible distributed platform architecture that connects these concerns into one consistent implementation.

I.4 Conclusion

This chapter established the industrial, business, and academic framing of AdOptimizer CDP. It clarified the role of SMART AUTOMATION TECHNOLOGIES, the positioning of the project within AdOptimizer AI, the nature of the underlying business problem, the motivation for a distributed lakehouse solution, the implemented objectives, and the limits of the present contribution. On this basis, the next chapter can now move from contextual framing toward the theoretical foundations that justify the architectural and engineering choices made in the project.

Chapter II

State of the Art and Theoretical Background

II.1 Introduction

This chapter establishes the theoretical foundation required to defend AdOptimizer CDP rigorously. The implemented platform is not merely a combination of tools; it is the result of architectural choices shaped by well-known constraints in distributed data engineering. For that reason, the chapter begins with general concepts and trade-offs before moving toward architectural models.

The discussion draws on the broader literature on distributed and data-intensive systems, including the design logic emphasized by Martin Kleppmann in *Designing Data-Intensive Applications*, especially around reliability, scalability, maintainability, partitioning, replication, and consistency trade-offs [15]. The objective is not to review the entire field, but to retain the concepts that directly justify the architecture adopted in this project.

II.2 Foundations of Modern Distributed Data Systems

II.2.1 From Centralized Systems to Distributed Data Platforms

Traditional information systems were largely organized around centralized databases and a relatively small number of application servers. This model remains effective when data volume is moderate, operational contexts are stable, and the number of consumers remains limited. However, once organizations begin collecting heterogeneous data from multiple

channels, centralized architectures become difficult to sustain. Storage pressure increases, computational workloads become more varied, and the system becomes more sensitive to single-node bottlenecks.

The centralized model is simple to deploy, but it concentrates ingestion, storage, processing, metadata, and access on the same execution point. As workloads grow, this concentration creates bottlenecks, single points of failure, and clear scalability limits.

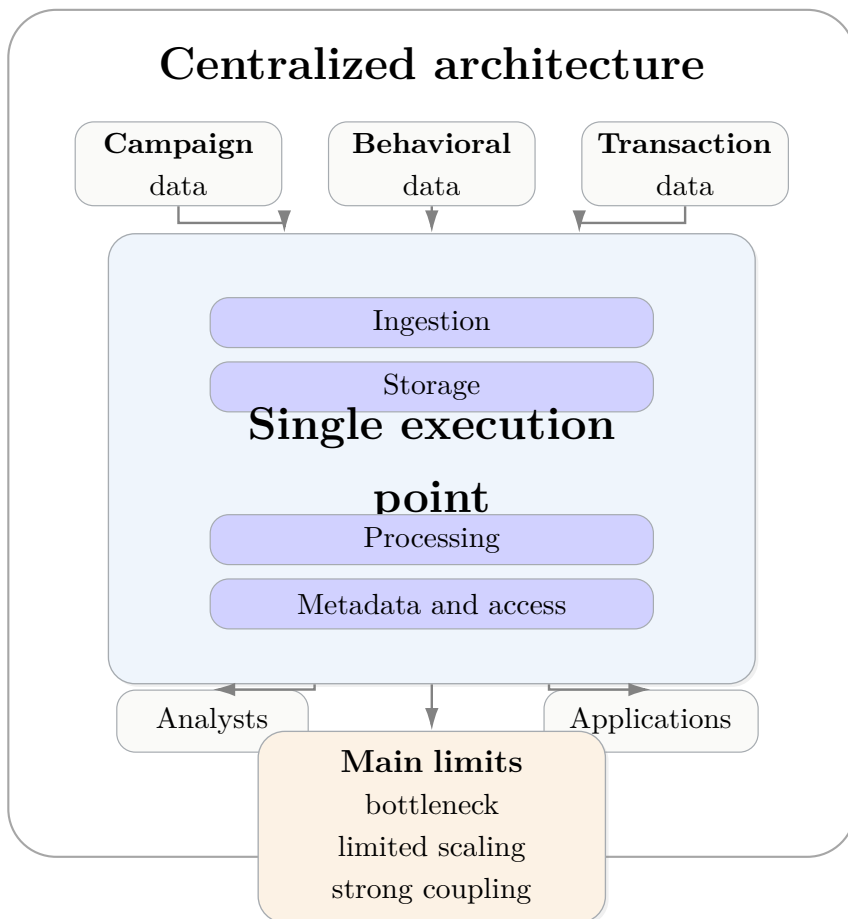


Figure II.1: Centralized architecture as a concentration of ingestion, storage, processing, and access on one execution point.

Figure II.1 highlights why centralized designs become fragile under heterogeneous analytical growth: all responsibilities compete on the same machine boundary, and operational evolution is constrained by the limits of that boundary.

Distributed data platforms emerged as a response to these limits. Their role is not only to increase capacity, but also to separate responsibilities such as ingestion, storage, compute, metadata, and analytical access. This decomposition allows each concern to evolve with the technical model best suited to it [15].

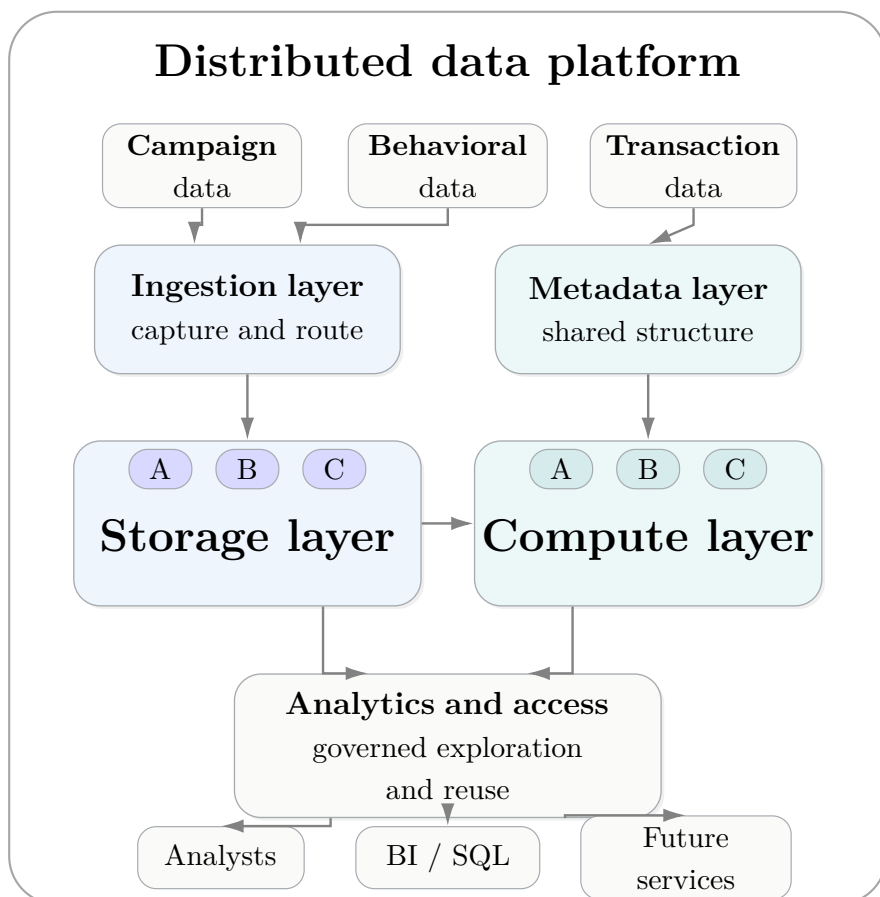


Figure II.2: Distributed data platform as a layered decomposition of storage, compute, metadata, and analytical access responsibilities.

By contrast, the distributed platform separates responsibilities according to operational role. This supports modular growth, parallel execution, better resilience, and clearer alignment between storage and processing patterns. Figure II.2 therefore prepares the discussion on horizontal scaling, reliability, and maintainability.

Criterion	Centralized architecture	Distributed data platform
Node organization	Core responsibilities remain concentrated on one primary execution point.	Responsibilities are distributed across several coordinated layers or nodes.
Scaling model	Capacity growth depends mainly on strengthening one machine.	Capacity growth is achieved by adding or specializing nodes and services.
Failure exposure	One overloaded or failed node can degrade the whole platform quickly.	Failure impact is more localized when replication and layered separation are used.
Data-processing logic	Ingestion, storage, processing, and access remain tightly coupled.	Ingestion, storage, compute, metadata, and access can evolve with different rhythms.
Governance evolution	Metadata and traceability are harder to extend cleanly as workloads diversify.	Governance can be attached explicitly to catalog, storage, and analytical layers.
Long-term suitability	More suitable for moderate and stable workloads.	More suitable for heterogeneous, growing, and multi-consumer analytical workloads.

Table II.1: Comparison between centralized and distributed architectural logic

II.2.2 Vertical Scaling vs Horizontal Scaling

One of the first decisions in large-scale systems concerns how growth should be absorbed. Vertical scaling strengthens a machine by increasing CPU, memory, or storage capacity. It is often simpler in the short term because the system does not need to be partitioned immediately. Nevertheless, it has two major limits: physical growth is finite, and dependence on a small number of powerful machines weakens resilience.

Horizontal scaling increases capacity by adding nodes. It is more complex because it requires workload distribution, coordination, and failure management. However, it offers better long-term elasticity and better alignment with modern data engineering workloads. For a distributed customer-data platform, horizontal scaling is therefore a structural architectural principle rather than a simple performance technique [15].

A simple way to express this intuition is:

$$C_{\text{total}} = n \times C_{\text{node}}$$

where n is the number of participating nodes and C_{node} is the average capacity provided by each node. The formula is deliberately simplified, because real systems include coordination overhead, but it captures the core architectural idea behind horizontal expansion.

II.2.3 Reliability, Scalability, and Maintainability

The literature on distributed systems frequently emphasizes three central qualities: reliability, scalability, and maintainability. Reliability refers to the system’s ability to continue providing correct service despite component failures, abnormal workloads, or operational disturbances. Scalability refers to the platform’s capacity to grow without disproportionate degradation or redesign. Maintainability refers to the ability to understand, operate, extend, and debug the system over time.

These qualities are fundamental for a platform such as AdOptimizer CDP. An unreliable platform cannot support trusted analytical reuse. A non-scalable platform cannot absorb new datasets or future use cases. A non-maintainable platform becomes fragile and difficult to operate. These qualities are therefore direct evaluation criteria for the engineering relevance of the project.

II.2.4 Data-Intensive Systems as a Distinct Engineering Problem

Data-intensive systems differ from ordinary software systems because data movement, storage, replayability, and consistency become first-class engineering concerns. In such systems, correctness is not defined only by the logic of business functions. It also depends on whether data is durably persisted, whether transformations are reproducible, whether schemas remain interpretable over time, and whether the system can explain the origin and lifecycle of the information it exposes.

This distinction is important for the positioning of the present work. AdOptimizer CDP is not simply an application with a database behind it. It is a distributed data engineering platform whose value lies in organizing the lifecycle of heterogeneous customer-related data in a reliable, observable, and governable way.

II.3 Core Theoretical Trade-offs in Distributed Data Engineering

II.3.1 The CAP Theorem and Consistency Trade-offs

The CAP theorem remains one of the best-known conceptual references in distributed systems. It states that under network partition conditions, a distributed system cannot simultaneously provide both strong consistency and full availability [12]. Although this statement is often simplified in practice, its educational importance is clear: once a system is distributed, architects must explicitly decide how consistency, availability, and failure

tolerance are balanced.

For data platforms, this means that different layers may adopt different trade-offs. Event transport, distributed storage, metadata services, and analytical table layers are not expected to behave identically. The value of CAP in the present context is therefore to justify why a multi-layer architecture is necessary.

II.3.2 ACID vs BASE Models

ACID properties traditionally define the transactional expectations of strongly controlled database systems: atomicity, consistency, isolation, and durability. These properties are essential when every update must remain immediately and strictly correct. However, not every stage of a distributed data platform requires the same guarantees at the same time.

BASE, commonly summarized as Basically Available, Soft state, Eventual consistency, represents a different philosophy. It accepts temporary divergence when this improves distribution or availability. Many ingestion layers operate closer to BASE-like behavior, whereas curated analytical layers benefit from stronger ACID-like semantics.

II.3.3 Data Partitioning and Replication

Partitioning divides data into smaller units so that storage and computation can be distributed across multiple nodes. It is one of the foundational mechanisms behind modern large-scale platforms. Without partitioning, a single node becomes a natural bottleneck for read throughput, write throughput, fault recovery, and transformation execution.

Replication complements partitioning by improving resilience and availability. When several copies of data or logs exist, the system can continue operating even if one node is unavailable. Yet replication also increases coordination and storage cost, which is why it must be treated as an architectural decision rather than a simple implementation detail.

The storage cost of replication can be summarized as:

$$S_{\text{effective}} \approx r \times S_{\text{raw}}$$

where S_{raw} is the original data volume and r is the replication factor. This relation shows why replication improves resilience while also increasing storage requirements.

II.3.4 Fault Tolerance and Service Resilience

In distributed systems, failure must be considered normal rather than exceptional. Nodes may disappear, ports may become temporarily unavailable, certificates may expire, services may start in the wrong order, and storage volumes may create cascading readiness issues. A resilient platform therefore needs explicit strategies for retries, health checks, replication, recovery, and replay.

This principle is especially relevant in data engineering because failures affect not only uptime but also trust in data products. A silent ingestion failure may create missing data, while a metadata failure may make tables unreadable. Fault tolerance is therefore directly tied to platform credibility and reproducibility.

II.3.5 Batch Processing vs Stream Processing

Batch processing is based on finite datasets processed periodically or on demand. It is suitable for historical backfills, curated analytical transformations, and reproducible large-scale recomputation. Stream processing is based on continuous flows of records or events that are handled incrementally as they arrive. It is useful when ingestion or propagation must remain closer to real-time behavior.

These two paradigms should not be treated as mutually exclusive. In modern platforms, they often coexist: streaming captures raw events continuously, while downstream computation structures them into trusted analytical tables through batch or micro-batch logic [18, 16].

At a very general level, processing intensity may be represented by:

$$\text{Throughput} = \frac{\text{Number of records processed}}{\text{Processing time}}$$

This expression is useful because it links theoretical processing models to an operational question: how much data can the platform absorb and transform within a given interval.

II.3.6 Schema-on-Write vs Schema-on-Read

Schema-on-write means that data must be shaped according to a predefined structure before it is stored in an authoritative analytical repository. This approach improves control and downstream semantic stability, but it may reduce ingestion flexibility. Schema-on-read, by contrast, stores raw or semi-structured data first and interprets structure later when the data is processed or queried.

This trade-off is central to modern lakehouse thinking. Raw layers benefit from schema-

on-read because they preserve source fidelity and replayability. Curated layers benefit from stronger schema discipline because users require stable analytical entities. A layered platform therefore combines both strategies rather than choosing only one.

II.4 Big Data Foundations

II.4.1 The Five Vs of Big Data

Big Data is often introduced through five dimensions: volume, velocity, variety, veracity, and value. Volume refers to the quantity of data that must be managed. Velocity refers to the rate at which data is generated and moved. Variety refers to the diversity of formats, structures, and source semantics. Veracity concerns the reliability and quality of the data. Value concerns the usefulness that can be extracted from it for decision-making and future intelligent services.

These dimensions remain relevant because they show that data engineering complexity does not depend only on scale. Even when volume is moderate, strong variety, reuse pressure, and governance expectations can justify a distributed architecture [17].

II.4.2 Heterogeneous Data Sources and Data Variety

Customer understanding rarely comes from a single operational system. It may involve campaign response datasets, online behavioral signals, e-commerce or retail transactions, session-level intent indicators, and profile-related attributes. Each source differs in format, cadence, completeness, and downstream meaning. This heterogeneity makes integration difficult because technical normalization is not sufficient; semantic reconciliation is also required.

As a result, variety becomes both a storage problem and a modeling problem. The platform must ingest different raw forms, persist them durably, and later organize them into analytical layers without erasing provenance.

II.4.3 Distributed Storage Principles

Distributed storage spreads persisted data across multiple nodes instead of concentrating it on a single machine. Its goals include capacity growth, resilience, replication, parallel read/write support, and better alignment with distributed compute. An important consequence of distributed storage is that it separates durable raw persistence from the compute jobs that later transform the data.

This separation has strong methodological value. When raw data is durably preserved, pipelines become replayable and easier to audit. Transformations can be rerun without repeatedly depending on the original upstream source.

II.4.4 Distributed Compute Principles

Distributed compute allows processing tasks to be decomposed and executed across several nodes. Instead of one machine reading and transforming an entire dataset, work is split according to partitions, files, or execution stages. This can reduce total execution time and align compute more closely with distributed storage [15].

However, distributed compute is beneficial only when the workload justifies it. Scheduling, serialization, dependency management, and cluster readiness introduce overhead. A defensible platform must therefore show that distributed compute is used where parallel execution is genuinely meaningful.

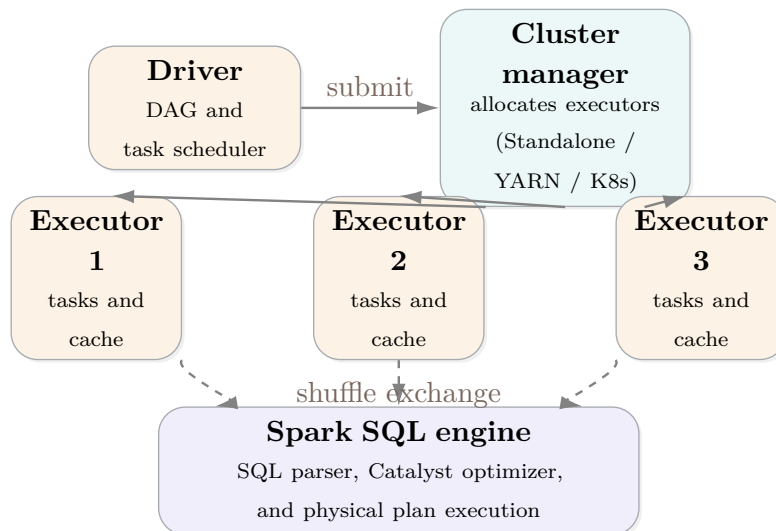


Figure II.3: Conceptual view of distributed execution in a Spark-style compute model.

Figure II.3 illustrates the operational logic behind this principle: a driver coordinates execution, a cluster manager allocates resources, executors process partitions in parallel, and the SQL engine materializes the physical plan that structures distributed analytical work.

II.5 Evolution of Analytical Data Architectures

II.5.1 Traditional Data Warehouse Architecture

The traditional data warehouse is organized around structured, curated, and strongly modeled analytical data. It is highly effective for stable reporting, dimensional modeling, and SQL-based decision support. Its main strength is semantic control.

Its limitation appears when heterogeneous or semi-structured data must be ingested rapidly and preserved before full normalization. In such contexts, the warehouse-only model can become too rigid because it expects analytical structure too early in the lifecycle.

II.5.2 Data Lake Architecture

The data lake emerged as a response to the rigidity of traditional warehouses. It enables large quantities of raw and semi-structured data to be stored before full modeling. This increases ingestion flexibility and supports exploratory reuse across different downstream consumers.

However, data lakes also introduced a risk: if metadata, naming discipline, and validation remain weak, the platform can degrade into an opaque storage zone with low analytical trust. The lake model solves flexibility problems but can create governance problems.

II.5.3 Data Lakehouse Architecture

The data lakehouse seeks to combine the flexibility of the lake with the transactional and governance discipline associated with warehouses. It preserves distributed and replayable storage logic while adding stronger table semantics, better schema evolution control, and improved analytical exposure [8].

This model is particularly well adapted to projects such as AdOptimizer CDP. It allows raw datasets to be preserved in distributed storage while curated datasets are exposed through governed tables suitable for transformation frameworks and SQL query engines.

II.5.4 Lambda Architecture

Lambda architecture organizes processing through two paths: a batch path for full historical recomputation and a speed layer for low-latency updates. Its objective is to reconcile correctness and freshness. The model was influential because it formalized the difficulty of satisfying both needs through a single pipeline.

Nevertheless, Lambda architecture is often criticized for operational duplication. Two processing logics must be maintained, monitored, and reconciled. Its academic relevance remains high because it helps explain why later architectures searched for simpler ways to preserve freshness without multiplying engineering burden.

II.5.5 Kappa Architecture

Kappa architecture reduces the conceptual duplication of Lambda by emphasizing a stream-centered approach in which the event log becomes the main source of truth. Reprocessing is achieved through replay rather than through a separate batch computation layer [16].

This model is attractive when event logs are rich and replay can serve as the main recomputation mechanism. However, many analytical platforms still require clearly governed historical layers and structured table abstractions. Kappa is therefore instructive, but not automatically sufficient for every customer-data use case.

II.5.6 Comparative Analysis of These Architectures

The historical movement from warehouses to lakes, from lakes to lakehouses, and from isolated reporting toward event-aware architectures reflects a broader shift in priorities. Early architectures privileged control and stable relational semantics. Later architectures privileged ingestion flexibility and scale. Contemporary platforms increasingly seek a hybrid answer that preserves raw fidelity, enables distributed processing, and still produces governed analytical products.

For AdOptimizer CDP, the lakehouse model is the most suitable. It supports raw bronze persistence, distributed storage, structured analytical tables, metadata-driven reuse, and downstream SQL exploration. The selected architecture is therefore a reasoned synthesis of the state of the art.

More precisely, the implemented project combines both Lambda-style and Kappa-style reasoning without reproducing either model in a pure textbook form. It remains Kappa-inspired because replayable incoming records and durable raw history are treated as operational assets rather than disposable transit. At the same time, it remains close to Lambda-style discipline because the main analytical semantics are stabilized through batch-oriented refinement over preserved historical data. The resulting architecture is therefore best understood as a governed hybrid lakehouse model: event-aware at ingestion time, but structurally batch-oriented when producing trusted analytical layers [18, 16, 8].

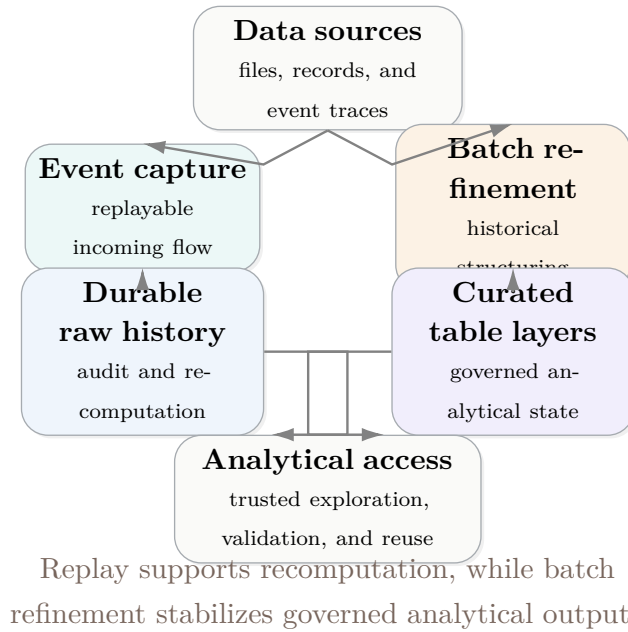


Figure II.4: Conceptual positioning of the implemented project between replay-oriented ingestion logic and batch-structured analytical refinement.

II.6 Foundational Concepts for Governance and Trust in Data Platforms

II.6.1 Metadata, Cataloging, and Data Discovery

Metadata gives meaning to stored data by describing structure, origin, semantics, and technical location. Without metadata, even accessible data can remain practically unusable. Cataloging extends this principle by making datasets discoverable and interpretable by different actors [15].

In a distributed customer-data platform, metadata is especially important because the architecture spans several services and layers. Governance cannot depend on informal memory alone. A defensible platform must therefore treat schema registration and dataset discovery as explicit capabilities.

II.6.2 Data Lineage and Traceability

Lineage refers to the ability to trace how data moved from source to output, which transformations were applied, and which downstream assets depend on which upstream artifacts. Traceability is essential for reproducibility, debugging, and auditability.

From an academic point of view, lineage helps distinguish a genuine data engineering

platform from a simple collection of disconnected scripts. When lineage can be reconstructed, the platform becomes explainable and easier to defend.

II.6.3 Data Quality Dimensions

Data quality is multidimensional. Common dimensions include completeness, consistency, validity, accuracy, timeliness, and uniqueness. These dimensions are especially important in customer-data contexts because downstream segmentation, profiling, or future predictive analytics become weak when source data is inconsistent or poorly governed.

Modern platforms increasingly formalize quality as an explicit layer of the architecture rather than as a purely manual cleaning activity. Checks, expectations, thresholds, and validation artifacts transform quality into a monitored engineering property.

For example, a simple completeness metric can be written as:

$$\text{Completeness} = \frac{\text{Valid records}}{\text{Expected records}}$$

Even such a basic ratio is valuable in practice because it transforms a qualitative concern into a measurable control signal that can be monitored over time.

II.6.4 Observability in Distributed Data Systems

Observability is the capacity to infer the internal state of a system through emitted evidence such as metrics, logs, traces, health checks, and runtime indicators. In distributed environments, observability is crucial because failures often emerge from the interaction between components rather than from one isolated service [21].

For data engineering, observability is tightly linked to platform trust. If the system cannot show whether ingestion succeeded, whether storage is healthy, whether services are reachable, or whether transformations produced expected results, its outputs become difficult to defend.

II.7 Security Foundations for Distributed Data Platforms

II.7.1 Authentication in Distributed Systems

Authentication establishes who a user or service is. In distributed environments, it becomes critical because communication crosses process, host, and network boundaries. Storage services, metadata layers, and query endpoints must be able to distinguish legitimate principals from unauthorized ones [19].

Without strong authentication, a distributed platform depends on implicit trust between components. Authentication is therefore a structural foundation of secure distributed data engineering.

II.7.2 Authorization and Access Control

Once identity is established, the system must determine which actions are permitted. Authorization controls access to services, datasets, administrative operations, and query interfaces. It operationalizes the principle of least privilege by restricting each actor to the minimum access necessary for its role.

In data platforms, authorization cannot be limited to end-user interfaces alone. It also applies to service-to-service calls, operational scripts, metadata access, and internal gateways.

II.7.3 Encryption in Transit and Encryption at Rest

Encryption in transit protects data while it moves between nodes, services, or users. It reduces exposure to interception and helps preserve confidentiality across the network. Encryption at rest protects persisted data on storage media and reduces exposure if storage devices or mounted runtime paths are accessed outside intended controls [9].

These two mechanisms answer different threat surfaces and must be treated as complementary. In a multi-service lakehouse platform, data moves through ingestion, orchestration, query, and monitoring channels, so both forms of protection are necessary.

II.7.4 Secure Service Exposure in Multi-Node Environments

Distributed platforms expose numerous service endpoints: orchestration UIs, query interfaces, storage consoles, monitoring dashboards, and service APIs. If these are exposed

without proxying, credential checks, and transport protection, the system accumulates a wide attack surface [20, 21].

Secure exposure therefore requires deliberate control of entry points, certificates, internal-versus-external interfaces, and service-access boundaries.

II.8 Conclusion

This chapter established the theoretical and architectural background that justifies the design of AdOptimizer CDP. It introduced the evolution of distributed data systems, the principal engineering trade-offs that shape them, the foundations of Big Data platforms, the comparative logic of modern analytical architectures, and the principles of governance, observability, and security in distributed environments.

On this basis, the following chapters can now move from theory to requirements, design, and implementation. The platform can therefore be presented not as a collection of tools, but as a coherent response to well-identified distributed data engineering challenges.

Chapter III

Technology Watch

III.1 Introduction

Chapter 2 established the conceptual logic behind distributed data platforms. This chapter moves to the concrete implementation layer by presenting the technologies retained for AdOptimizer CDP. The goal is not to produce exhaustive product documentation, but to explain why each selected tool deserves its place in the final platform and how the stack becomes coherent when these tools are assembled together.

III.2 Selection Criteria

- **Distributed compatibility:** the tool had to fit a multi-node platform rather than a local-only demonstration.
- **Functional relevance:** each retained component had to answer a clear need in ingestion, storage, processing, governance, quality, orchestration, or observability.
- **Integration coherence:** the stack had to remain technically consistent across services and interfaces.
- **Operational defensibility:** the selected technologies had to support reproducibility, monitoring, and service separation.
- **Academic readability:** the final choices had to remain understandable and explainable in a final-year engineering report.



Figure III.1: Technology retained for event ingestion and topic-based transport

III.3 Apache Kafka

- **Role in the platform:** Apache Kafka acts as the ingestion backbone between dataset producers and downstream storage services [5].
- **Reason for selection:** it structures raw data transport through explicit topics and allows controlled replay when a stage must be re-executed.
- **Value for the project:** Kafka makes ingestion observable, modular, and better aligned with modern event-driven data engineering practices than direct one-shot file transfers.

III.4 Hadoop HDFS



Figure III.2: Technology retained for distributed bronze-layer storage

- **Role in the platform:** HDFS stores the bronze data generated from Kafka consumption [2].
- **Reason for selection:** it offers distributed persistence, replication, and strong compatibility with the surrounding Hadoop and Spark ecosystem.
- **Value for the project:** HDFS gives the platform a real distributed storage layer instead of a local folder-based approximation, which strengthens the realism of the deployment.

III.5 Apache Iceberg and Hive Metastore

- **Role in the platform:** Iceberg supports curated table layers, while Hive Metastore centralizes their metadata [4, 3].



Figure III.3: Technologies retained for governed lakehouse tables and metadata management

- **Reason for selection:** Iceberg provides a modern table abstraction suited to structured analytics, and Hive Metastore remains a practical catalog layer understood by several engines.
- **Value for the project:** together they transform simple file persistence into a genuine lakehouse organization where curated data can be queried, documented, and evolved more cleanly.

III.6 Apache Spark



Figure III.4: Technology retained for distributed processing and analytical loading

- **Role in the platform:** Spark is the main distributed compute engine used for transformation and loading tasks [7].
- **Reason for selection:** it integrates naturally with HDFS, Iceberg, and the cluster deployment model, while remaining academically recognizable as a standard big-data processing engine.
- **Value for the project:** Spark gives the platform real parallel computation capabilities and serves as the backbone for both the raw-to-curated flow and the lakehouse loading logic.

III.7 Trino



Figure III.5: Technology retained for interactive SQL access

- **Role in the platform:** Trino exposes the analytical tables through an SQL query interface [22].
- **Reason for selection:** it separates interactive exploration from heavy transformation workloads and provides a clean querying layer for validation and analysis.
- **Value for the project:** Trino improves usability, supports manual verification through SQL clients, and reinforces the idea that a lakehouse must serve both engineering and analytical access needs.

III.8 dbt for Transformation Modeling



Figure III.6: Technologies retained for model-driven transformations and analytical storage format support

- **Role in the platform:** dbt structures the staging, intermediate, and analytics transformations as explicit models [10].
- **Reason for selection:** it brings readability, dependency management, and reproducibility to the transformation layer.
- **Why Parquet is mentioned with it:** Parquet remains an efficient columnar analytical format that naturally complements curated transformation outputs [6].
- **Value for the project:** dbt turns transformations into governed assets rather than ad hoc scripts, which makes the platform easier to explain, validate, and maintain.

III.9 Great Expectations



Figure III.7: Technology retained for explicit data quality validation

- **Role in the platform:** Great Expectations formalizes validation checks on important data assets [14].

- **Reason for selection:** it helps transform quality from an implicit assumption into a visible engineering control.
- **Value for the project:** the tool provides auditable validation artifacts and strengthens the trustworthiness of the resulting data products.

III.10 Apache Airflow



Figure III.8: Technology retained for workflow orchestration

- **Role in the platform:** Airflow orchestrates the ordered execution of reset, ingestion, transformation, validation, and lakehouse readiness workflows [1].
- **Reason for selection:** it provides explicit DAG-based orchestration, retries, scheduling, and operational visibility.
- **Value for the project:** Airflow transforms the platform from a collection of scripts into a controlled execution system with traceable runs and clear dependencies.

III.11 Prometheus and Grafana



Prometheus



Grafana

Figure III.9: Technologies retained for observability and dashboarding

- **Role in the platform:** Prometheus collects technical metrics, while Grafana displays them through operational dashboards [21, 13].
- **Reason for selection:** the pair is widely used, easy to justify academically, and especially useful in distributed environments where service state must remain visible.
- **Value for the project:** observability improves the platform's credibility by making resource health, service exposure, and pipeline behavior measurable rather than assumed.

III.12 Docker

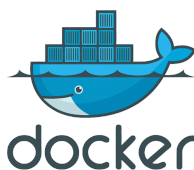


Figure III.10: Technology retained for service isolation and reproducible deployment

- **Role in the platform:** Docker isolates services and makes the deployment reproducible across the control plane and VM nodes [11].
- **Reason for selection:** it simplifies service packaging, environment consistency, and deployment repetition.
- **Value for the project:** containerization makes the platform easier to rebuild, demonstrate, and stabilize, which is especially important in an academic distributed setup.

III.13 Synthesis of the Final Stack

- Kafka structures ingestion.
- HDFS persists the bronze layer in distributed storage.
- Iceberg and Hive Metastore organize the lakehouse tables and metadata.
- Spark executes the main distributed processing logic.
- Trino exposes interactive analytical querying.
- dbt governs transformations as explicit data models.
- Great Expectations formalizes data quality checks.
- Airflow orchestrates the end-to-end execution order.
- Prometheus and Grafana expose observability.
- Docker preserves deployment reproducibility and service isolation.

III.14 Conclusion

This chapter clarified the technological foundation of AdOptimizer CDP by structuring the stack around the concrete tools used in the implementation. Each retained component answers a precise role in the platform, and the value of the solution comes from the coherence of these roles rather than from the isolated adoption of popular technologies. With this technical watch now clarified, the report can move toward requirements analysis and system design.

Chapter IV

Requirements Analysis and System Design

IV.1 Introduction

This chapter translates the business framing and technical choices established in the previous chapters into a structured design baseline for AdOptimizer CDP. Its purpose is to identify what the platform must do, which constraints it must respect, who will interact with it, and how the final multi-layer architecture answers those needs. Unlike Chapter 3, which focused on tool selection, this chapter focuses on requirement formalization and on the architectural organization of the implemented solution. In practice, the design logic follows the same principles emphasized in modern data-intensive system engineering: clear separation of concerns, explicit data movement boundaries, and layered coordination between storage, compute, and access services [15, 8].

IV.2 Requirement Elicitation Context

The requirements of AdOptimizer CDP were derived from three complementary sources:

- the business objective of creating a distributed customer-data foundation under the broader AdOptimizer AI vision;
- the engineering need to ingest, store, process, validate, orchestrate, expose, and monitor heterogeneous customer-related datasets;
- the operational reality of a lightweight academic multi-node environment with security hardening requirements.

As a result, the chapter does not present abstract requirements in isolation. Each requirement is directly connected to an implemented capability of the platform and to the final distributed deployment model.

IV.3 Functional Requirements

The functional scope of the platform is centered on the end-to-end lifecycle of customer-related data, from ingestion to governed analytical access. The most important functional requirements are summarized in Table IV.1 and Table IV.2.

ID	Functional requirement	Technical acceptance criterion
FR-1	Ingest heterogeneous customer-related datasets through a structured pipeline.	The platform accepts the active Client 1 datasets and routes them through the defined ingestion workflow without manual reshaping of the overall process.
FR-2	Provide topic-based transport for reproducible raw data movement.	Kafka topics are created, used, and reset through controlled pipeline operations, with visible topic-based message flow.
FR-3	Persist replayable raw data in a distributed bronze landing zone.	HDFS stores bronze artifacts in a durable, distributed structure that can be re-read by downstream Spark processing stages.
FR-4	Load bronze data into governed raw lakehouse tables.	Spark reads the persisted bronze files and writes structured raw tables registered in the lakehouse layer.
FR-5	Centralize analytical metadata for table discovery and reuse.	Hive Metastore exposes the table definitions required for downstream engines to identify and access the lakehouse objects.

Table IV.1: Primary functional requirements of the platform, part 1

ID	Functional requirement	Technical acceptance criterion
FR-6	Provide interactive SQL access for analytical exploration.	Trino executes catalog, schema, row-count, and representative analytical queries over governed lakehouse tables.
FR-7	Support layered transformations from raw to curated analytical outputs.	dbt-spark runs declared transformation models and preserves dependency-aware model execution.
FR-8	Apply lightweight data quality validation and persist validation evidence.	Great Expectations checks structural and semantic rules and records validation artifacts for the controlled datasets.
FR-9	Orchestrate the operational sequence of the data platform.	Airflow executes environment reset, readiness, ingestion, and validation DAGs with visible task state and run history.
FR-10	Expose infrastructure and pipeline observability signals.	Prometheus scrapes the configured endpoints and Grafana displays service-readiness and pipeline-oriented monitoring indicators.

Table IV.2: Primary functional requirements of the platform, part 2

IV.4 Non-Functional Requirements

Beyond functional behavior, the platform must also satisfy a set of quality constraints related to performance, modularity, traceability, security, and demonstration readiness. These requirements are summarized in Table IV.3. Their importance is consistent with the broader literature on reliable and maintainable distributed systems, in which reproducibility, observability, and controlled failure handling are treated as architectural qualities rather than optional conveniences [15].

ID	Non-functional requirement	Design implication
NFR-1	Scalability through distributed execution rather than single-node concentration.	Storage, compute, and query roles are separated across multiple nodes and services.
NFR-2	Modularity and separation of concerns.	Ingestion, storage, processing, metadata, validation, orchestration, monitoring, and security are implemented as distinct layers.
NFR-3	Reproducibility of ingestion and transformation flows.	The platform preserves replayable bronze data, versioned transformation logic, and explicit orchestration sequences.
NFR-4	Observability of service and pipeline behavior.	Health checks, metrics, dashboards, and orchestration logs must make failures visible and diagnosable.
NFR-5	Security of access, movement, and persisted runtime data.	Authentication, encrypted service exposure, and protected storage must be integrated into the deployment design.
NFR-6	Academic clarity and demonstrability.	The architecture must remain explainable, structured, and feasible to defend in front of a jury.

Table IV.3: Main non-functional requirements of the platform

IV.5 Global Logical Architecture

At the logical level, AdOptimizer CDP is organized as a layered customer-data platform rather than as a monolithic data pipeline. This organization is aligned with lakehouse thinking, where raw persistence, structured table layers, metadata governance, and analytical access are coordinated but not collapsed into one opaque runtime block [8]. Each layer has a precise responsibility in the end-to-end flow:

- **Ingestion layer:** receives and transports the raw input datasets through Kafka.
- **Bronze persistence layer:** stores replayable raw artifacts in HDFS.
- **Raw lakehouse layer:** structures the raw persisted inputs through Spark and Iceberg.
- **Metadata layer:** centralizes table definitions through Hive Metastore.

- **Analytical access layer:** exposes curated tables through Trino.
- **Transformation layer:** applies modeled transformations through dbt-spark.
- **Validation layer:** checks trust and structural integrity through Great Expectations.
- **Orchestration and observability layer:** coordinates and supervises the full process through Airflow, Prometheus, and Grafana.

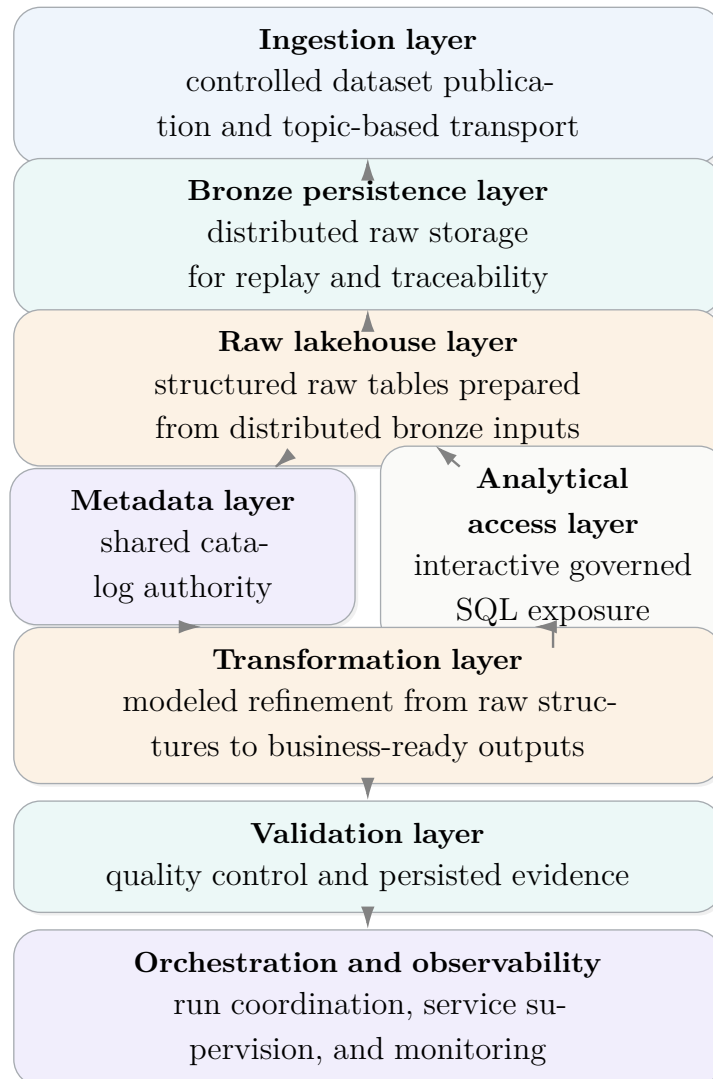


Figure IV.1: Global logical architecture of the platform from ingestion to analytical access, validation, and orchestration.

This logical decomposition is important because it explains the system as a coordinated architecture rather than as a sequence of unrelated tools. It also clarifies why the same dataset may pass through different representations before becoming a trusted analytical asset.

IV.6 Physical Multi-Node Architecture

At the physical level, the final design is distributed across one control plane and three data-plane virtual machines. Table IV.4 summarizes this allocation.

Node	IP / Type	VM specifications	Main services	Architectural role
Control plane	Local workstation	Not applicable, because this node is not part of the Linux VM fleet.	Airflow, Prometheus, Grafana, Kafka UI, metrics exporter, local TLS gateway, remote orchestration runtime	Supervises, orchestrates, monitors, and drives the distributed data plane.
VM1	10.10.252.11	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 1, HDFS DataNode 1, Spark worker 1, Trino worker 1, cAdvisor	Worker node dedicated to distributed storage, processing, and query participation.
VM2	10.10.252.12	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 2, Kerberos KDC, NameNode, DataNode 2, Hive Metastore, Spark master, Spark submit helper, Spark worker 2, Spark Thrift Server, Trino coordinator, HTTPS Trino gateway, cAdvisor	Leader node of the data plane and central coordination point for protected cluster services.
VM3	10.10.252.13	Ubuntu 22.04.5 LTS; QEMU virtual CPU (2 sockets $\times$ 2 cores); 3.8 GiB RAM; 26 GB disk; no dedicated GPU.	Kafka broker 3, HDFS DataNode 3, Spark worker 3, Trino worker 2, cAdvisor	Additional worker node reinforcing distributed storage, compute, and query execution.

Table IV.4: Physical node distribution of the final deployment

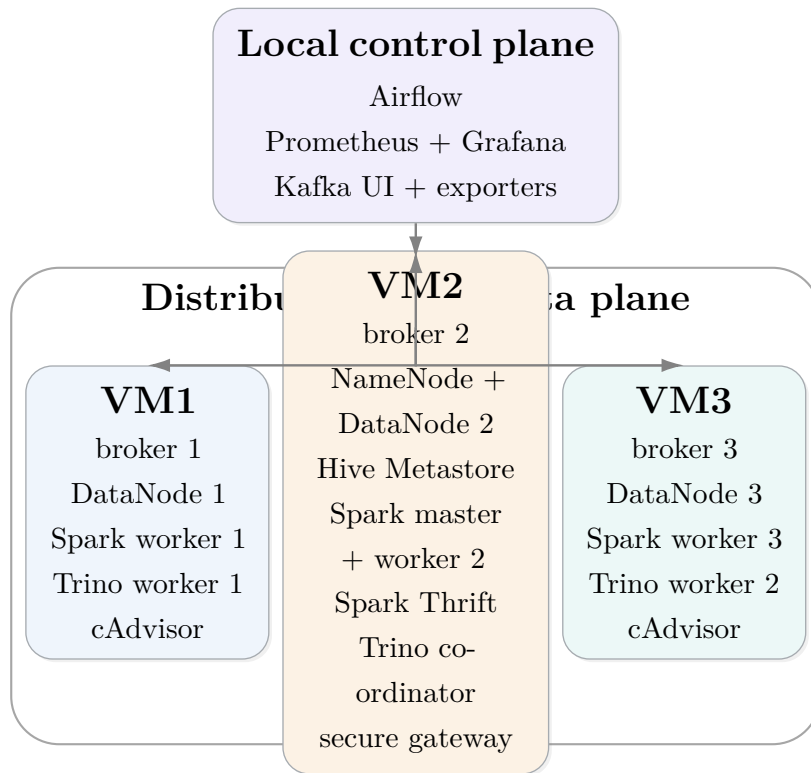


Figure IV.2: Physical multi-node architecture showing the local control plane and the service placement across VM1, VM2, and VM3.

IV.7 End-to-End Data Flow

The end-to-end operational flow of the platform can be summarized in the following sequence:

1. source datasets are prepared and published through the ingestion workflow;
2. Kafka receives and organizes the movement through controlled topics;
3. bronze artifacts are landed in HDFS as replayable raw persistence;
4. Spark reads the bronze zone and loads structured raw tables into the lakehouse;
5. Hive Metastore registers the table definitions;
6. Trino exposes analytical access to the governed tables;
7. dbt-spark applies business-oriented transformations;
8. Great Expectations validates the resulting datasets;
9. Airflow orchestrates and supervises the sequence;

10. Prometheus and Grafana expose the monitoring evidence.

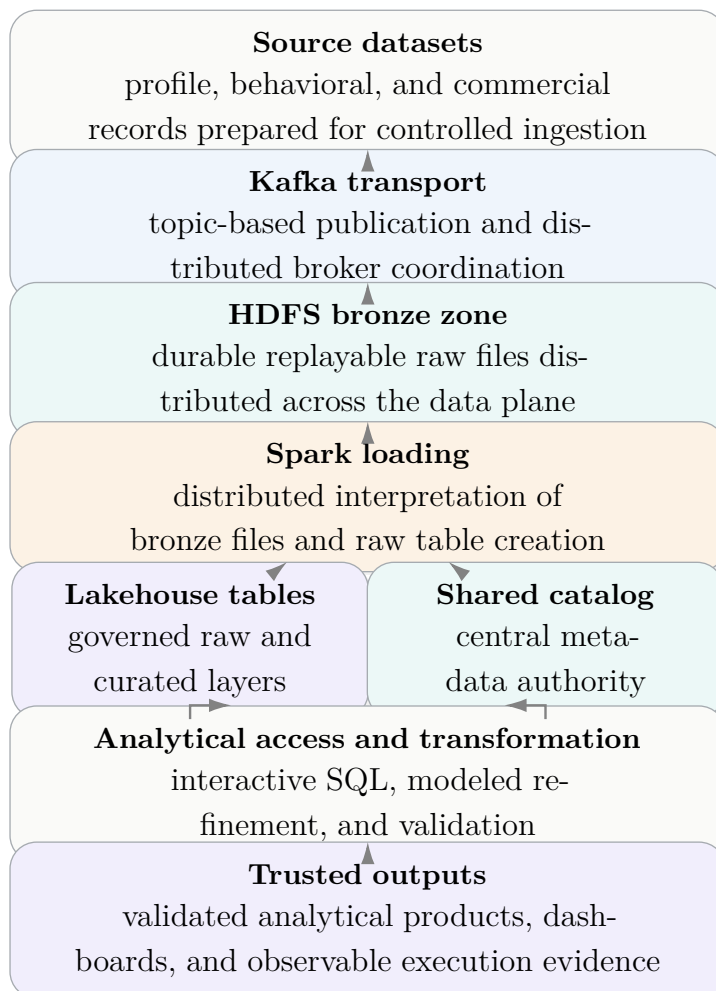


Figure IV.3: End-to-end flow from heterogeneous source datasets to validated analytical outputs.

This sequence is especially important for the report because it gives the jury a simple operational story to follow when moving from requirements to implementation and later to validation.

IV.8 Security-by-Design Integration

Security is not positioned as an external afterthought. It is integrated into the design of the platform through explicit controls attached to core layers. This follows the same engineering rationale as the rest of the architecture: identity, traffic protection, and protected storage boundaries are treated as part of the system design itself, not as a late operational patch [2, 19, 9]. Table IV.5 summarizes this logic.

Security control	Protected scope	Design contribution
Kerberos	Hadoop secure mode and service authentication	Ensures that protected cluster services rely on explicit identity rather than implicit trust.
SPNEGO	Secured Hadoop web interfaces	Extends authenticated access to browser-facing Hadoop endpoints.
TLS	Sensitive exposed service endpoints	Protects data movement and browser-facing access paths.
LUKS	Runtime storage volumes on the VMs	Reinforces encryption at rest for persisted service data.
SSH-based remote orchestration	Control-plane to data-plane submission path	Keeps remote execution structured and controlled from the orchestration side.
Authenticated gateways	Query and access entry points	Reduces uncontrolled direct exposure of critical services.

Table IV.5: Security controls integrated into the system design

IV.9 Design Rationale and Trade-offs

The final design is the result of deliberate engineering trade-offs rather than maximum-complexity choices:

- **Distributed proof over oversized scale:** the architecture demonstrates real service distribution while remaining feasible on lightweight academic VMs.
- **Layering over monolithic simplification:** the system is decomposed into clear responsibilities to strengthen maintainability and defense clarity.
- **Security hardening without architectural replacement:** the security layer reinforces the platform without changing its lakehouse logic.
- **Operational clarity over hidden automation:** explicit scripts, bundles, checks, and service roles make the deployment easier to explain and debug.
- **Future readiness over premature AI claims:** the design prepares downstream Customer 360 and AI use cases without pretending that those intelligent modules are already fully implemented.

These trade-offs are important for the jury defense because they show that the architecture was shaped by real constraints, not by arbitrary tool accumulation.

IV.10 Conclusion

This chapter converted the business framing and technical stack into a coherent design baseline for AdOptimizer CDP. It formalized the main functional and non-functional

requirements, described the global logical architecture, clarified the physical multi-node deployment, summarized the end-to-end data flow, and positioned security as an integrated design concern.

On this basis, the next chapter can now present the concrete implementation of the distributed lakehouse platform, using the design choices defined here as the reference model.

Chapter V

Implementation of the Distributed Lakehouse Platform

V.1 Introduction

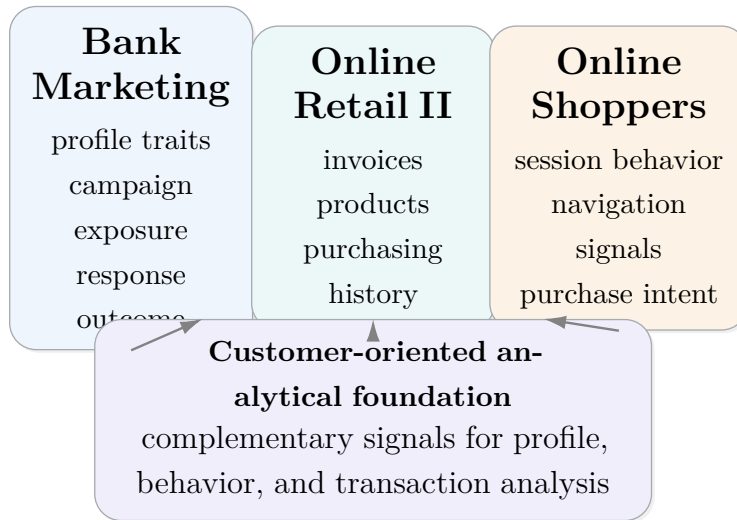
This chapter presents the concrete implementation of AdOptimizer CDP as an operational distributed lakehouse platform. While Chapter 4 defined the design baseline, the present chapter explains how that design was translated into working services, executable pipelines, governed data layers, and secured access paths. The objective is not to describe isolated tools separately, but to show how ingestion, storage, processing, metadata, querying, transformation, quality validation, and orchestration were assembled into one coherent engineering system.

V.2 Data Sources and Dataset Modeling

The implementation is based on a compact but representative dataset set selected to support the Customer 360 orientation of the platform while remaining compatible with the available academic infrastructure. Rather than using one oversized monolithic source, the project retains several complementary datasets, each contributing a different view of customer behavior.

Each dataset keeps its source identity during ingestion and bronze persistence. This choice is important because the platform is intended to preserve replayability and traceability before any normalization or business transformation is applied. In practice, the implementation therefore distinguishes between source-specific raw inputs and later converged analytical models. This allows the platform to defend both schema fidelity at the entry point and business-oriented integration in upper layers.

Dataset	Analytical role	Customer signal covered	Implementation contribution
Bank Marketing	campaign and response analysis	contact history, subscription outcome, profile attributes	provides customer profile and campaign-oriented raw inputs
Online Retail II	transactional and commercial behavior analysis	invoices, products, quantities, revenue, countries	provides purchase history and product interaction facts
Online Shoppers Purchasing Intention	digital behavioral analysis	session behavior, navigation, purchase intention	provides pre-purchase intent and online behavior indicators

Table V.1: Active datasets retained in the implemented platform**Figure V.1:** Active dataset families and their complementary contribution to the customer-oriented analytical scope of the platform.

V.3 Kafka-Based Ingestion Layer

Kafka is used as the transport backbone of the ingestion layer. The implementation does not load datasets directly into storage or processing services; instead, each source is first published through a topic-oriented flow that decouples dataset reading from downstream persistence. This approach improves control, replayability, and operational clarity, which is consistent with both the Kafka event-log model and broader stream-oriented architectural reasoning [5, 16].

The Kafka cluster is distributed across the three virtual machines:

- broker 1 on VM1;
- broker 2 on VM2;
- broker 3 on VM3.

At the implementation level, each source dataset is associated with its own topic family. This prevents unrelated sources from being merged too early and makes reset and replay operations more explicit. The control plane triggers the ingestion logic, producers publish the records, and downstream consumers persist the bronze output in HDFS. This means Kafka acts as the controlled transport boundary between local source preparation and distributed storage.

The platform also implements topic reset logic within the orchestration flow. This is important for demonstrations and reproducible reruns, because it allows the environment reset pipeline to clean old ingestion traces before a fresh execution begins.

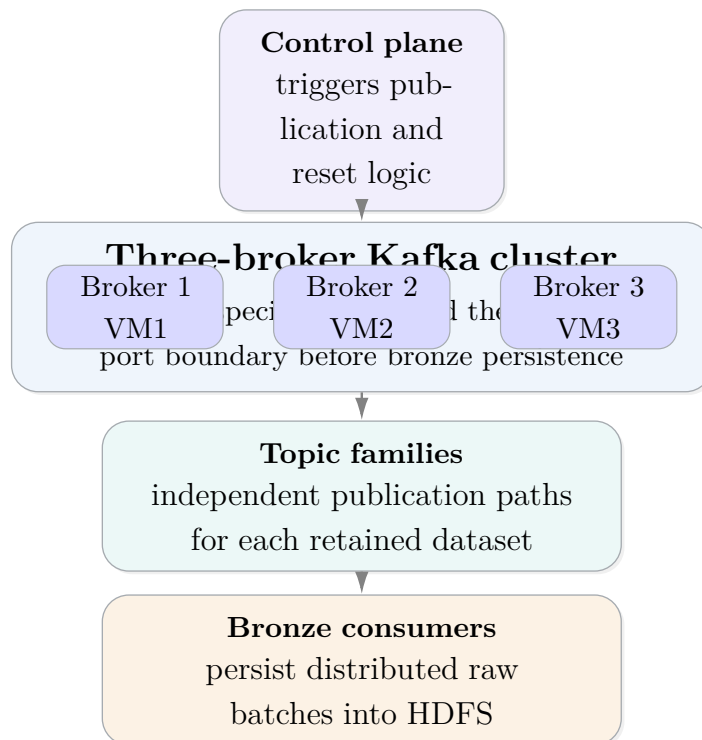


Figure V.2: Dataset publication flow from the control plane to the three-broker Kafka cluster and toward bronze persistence.

V.4 HDFS Bronze Storage Layer

The bronze layer is implemented on HDFS as the replayable raw persistence zone of the platform. After Kafka-based transport, source records are landed into HDFS in batch-oriented bronze files before any lakehouse structuring step is executed. This choice gives the project an explicit raw persistence boundary, which is essential for reproducibility, debugging, and future reprocessing.

The HDFS cluster is implemented with:

- the NameNode on VM2;

- one DataNode on each VM: VM1, VM2, and VM3.

In operational terms, this means the namespace authority remains centralized on VM2, while block storage is distributed across the three nodes. The implemented bronze organization is client-oriented and source-aware: each dataset is persisted under a controlled bronze structure rather than being mixed into one generic folder. This reinforces traceability and keeps the raw layer easy to inspect.

The bronze layer also plays a strategic role in the whole project: it is the stable checkpoint between ingestion and compute. If a downstream Spark stage fails, the platform does not need to reread the original local files or republish the full datasets manually. It can simply restart from the bronze persistence already distributed in HDFS.

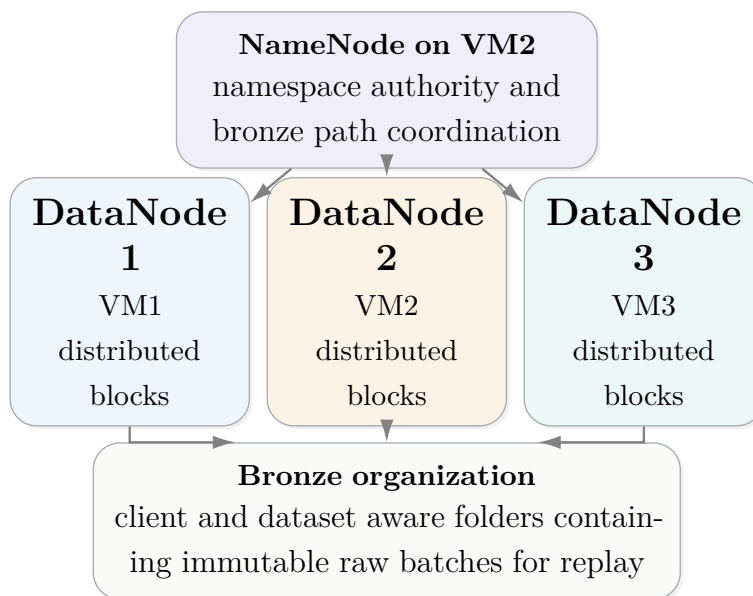


Figure V.3: Bronze storage organization in HDFS, with NameNode coordination on VM2 and DataNode distribution across VM1, VM2, and VM3.

V.5 Spark Processing and Iceberg Loading

Spark is the main distributed processing engine of the platform. Its implementation is aligned with the physical cluster design and with the lakehouse objective of turning replayable distributed files into governed analytical tables [7, 4, 8]:

- the Spark master runs on VM2;
- workers are distributed across VM1, VM2, and VM3;
- the control plane submits jobs remotely to the Spark environment on VM2.

This execution path is important. The final deployment does not rely on a heavy local Spark driver on the workstation. Instead, the control plane orchestrates remote execution through the distributed cluster, which is closer to a production-style separation between orchestration and compute.

From a data perspective, Spark reads the bronze files from HDFS, interprets their schemas, normalizes the raw structures, and writes the outputs into Iceberg-backed raw lakehouse tables. The implementation therefore transforms replayable raw artifacts into governed table objects while preserving distributed execution. The raw lakehouse layer is not yet the final curated business layer; it is the structured foundation on top of which transformations and validations are later applied.

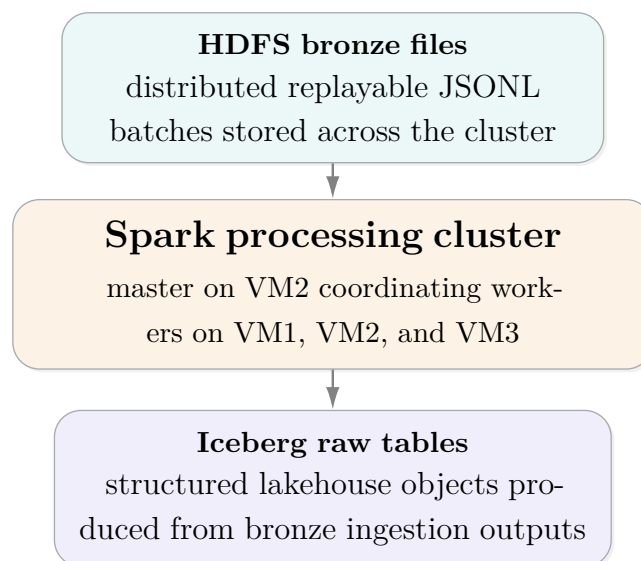


Figure V.4: Distributed Spark raw-load path from replayable bronze batches to governed raw lakehouse tables.

V.6 Hive Metastore and Catalog Governance

Hive Metastore is implemented as the shared metadata authority of the lakehouse platform. It is deployed on VM2 together with its PostgreSQL backing store and acts as the central catalog reference for upper services such as Spark and Trino [3, 4].

Its role in the implementation is essential: once Spark writes the raw Iceberg tables, their definitions must be discoverable through a common metadata layer. Without this shared governance service, each engine would have to manage table visibility independently, which would weaken consistency and make the architecture harder to defend.

In practical terms, Hive Metastore allows the platform to:

- register the lakehouse tables created after Spark loading;

- expose the same logical objects to different access engines;
- preserve a governed catalog boundary between raw persistence and analytical consumption.

This makes the metadata layer a real implementation component, not a theoretical addition. It is the reason the lakehouse can behave like a coordinated data platform rather than a simple collection of files.

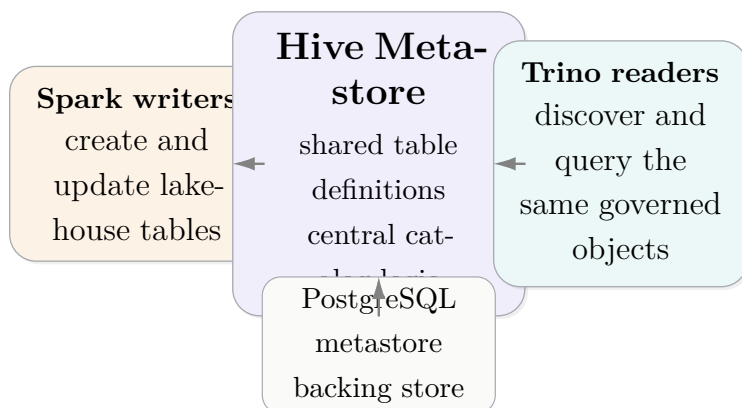


Figure V.5: Shared metadata governance through Hive Metastore between distributed writers and analytical readers.

V.7 Trino Query Layer

Trino provides the analytical SQL access layer of the platform. Its implementation follows a coordinator/worker separation consistent with distributed query-engine design [22]:

- the coordinator is hosted on VM2;
- worker 1 is hosted on VM1;
- worker 2 is hosted on VM3.

This arrangement is important because it demonstrates that the query layer is also distributed, not only the storage or compute layer. Trino consumes the shared catalog definitions and exposes interactive SQL access over the Iceberg tables. In the project, this layer is used for readiness checks, lakehouse verification, dataset inspection, and analytical exploration.

The query access path is also protected by an authenticated gateway. As a result, the Trino browser interface is not exposed as an uncontrolled plain endpoint. This reinforces the idea that access services in the final platform must remain demonstrable but still aligned with the security posture of the deployment.

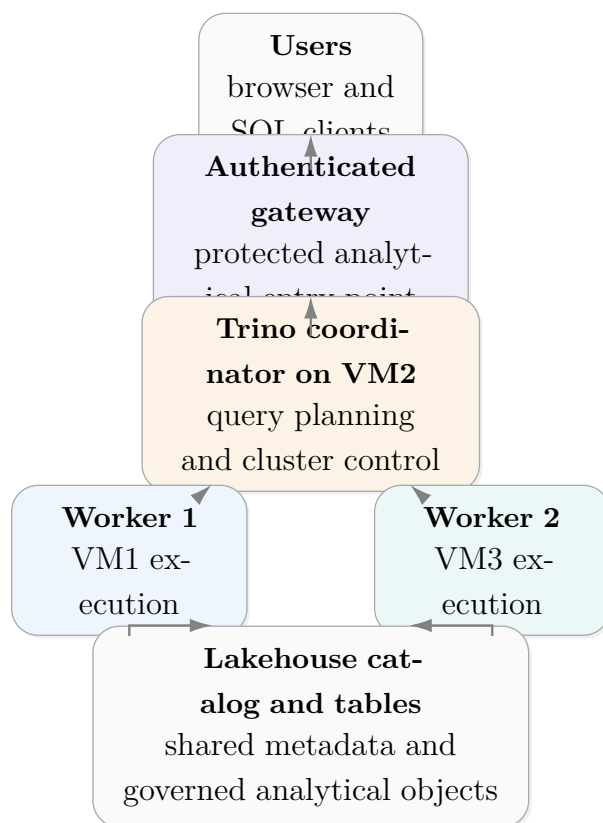


Figure V.6: Protected Trino analytical access path from secure entry point to coordinator and distributed workers.

V.8 dbt-spark Transformation Layer

dbt-spark is used to organize transformations as declarative models rather than as ad hoc processing scripts. This implementation choice is important because it introduces structure, dependency control, and documentation logic into the transformation stage [10].

The implemented transformation flow follows a layered modeling philosophy:

- raw lakehouse tables remain the direct output of the Spark loading stage;
- staging models standardize and clean the raw structures;
- intermediate models prepare reusable analytical logic;
- analytical models expose business-oriented outputs ready for exploration.

In this design, dbt does not replace Spark. Instead, it builds on top of Spark Thrift access and uses the distributed lakehouse environment as its execution target. This distinction matters in the report because it shows that transformation governance is separated from raw compute mechanics: Spark provides the engine, while dbt provides the modeling discipline.

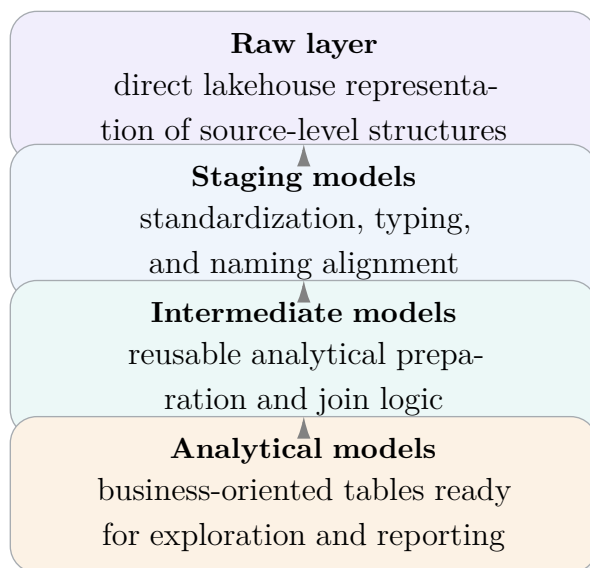


Figure V.7: Layered dbt transformation flow from raw tables to staging, intermediate, and analytical outputs.

V.9 Great Expectations Validation Layer

Great Expectations is implemented as the lightweight validation layer of the platform. Its role is not to perform generic visual checking, but to formalize trust conditions over the produced data. This is especially important in an academic lakehouse platform, where the project must prove that outputs are not only generated, but also controlled [14].

The implemented validation logic targets the structural and semantic integrity of selected lakehouse layers. In practice, the platform uses Great Expectations to check that the resulting data products remain consistent with expected shapes and business-level constraints. Validation artifacts are then persisted so that pipeline runs can be defended with observable evidence instead of informal claims.

This layer is therefore central to the governance dimension of the project. It strengthens the transition from “data processed successfully” to “data can be considered trustworthy enough for analytical use.”

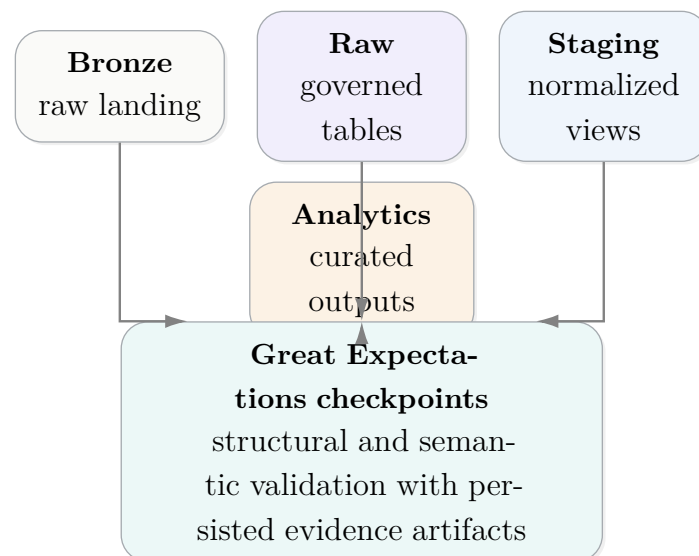


Figure V.8: Validation checkpoints over the lakehouse layers and the persistence of validation evidence.

V.10 Airflow Orchestration Layer

Airflow orchestrates the platform from the local control plane. In the final implementation, orchestration is not reduced to one oversized DAG. Instead, the project is structured around a small set of operational DAGs, each with a clear responsibility in the lifecycle of the environment [1].

DAG	Operational role	Implementation value
Environment reset pipeline	cleans Kafka, HDFS bronze traces, and lakehouse state	enables reproducible reruns from a controlled baseline
Lakehouse setup pipeline	prepares technical structures and service prerequisites	ensures the catalog and storage environment are initialized consistently
Lakehouse readiness pipeline	validates that the main services are reachable and usable	prevents execution from starting on an unstable cluster
Kafka–HDFS–Spark lakehouse pipeline	executes the raw ingestion and loading flow	materializes the raw distributed lakehouse foundation
dbt-spark lakehouse pipeline	runs modeled transformations and validation-oriented upper steps	produces curated analytical outputs on top of the raw layer

Table V.2: Implemented orchestration flow of the platform

This orchestration strategy is important because it matches the real operational logic of the project. The platform can be reset, checked, loaded, transformed, and validated in separate controlled stages. It also makes failures easier to diagnose, because a problem can be linked to a specific orchestration boundary instead of being hidden inside one monolithic workflow.

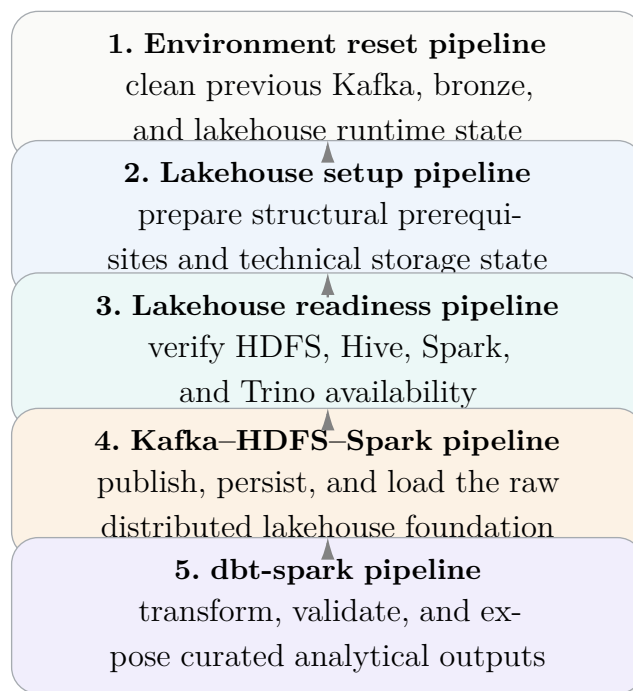


Figure V.9: Five-DAG orchestration sequence from environment preparation to raw loading and curated transformation.

V.11 Conclusion

This chapter showed how the design baseline of the previous chapter was turned into a concrete distributed implementation. The platform now appears as a full engineering chain: heterogeneous customer datasets are published through Kafka, persisted in HDFS bronze storage, processed through Spark, structured into Iceberg tables, governed through Hive Metastore, queried through Trino, transformed through dbt-spark, validated through Great Expectations, and orchestrated through Airflow.

The implementation therefore confirms that the project is not a simple tool collection. It is a layered, multi-service, distributed lakehouse platform with explicit operational logic and clear data movement boundaries. On this basis, the next chapter can focus on deployment strategy and distributed execution perspective, explaining how this implementation is physically organized and operated across the final multi-node environment.

Chapter VI

Deployment Strategy and Distributed Execution

VI.1 Introduction

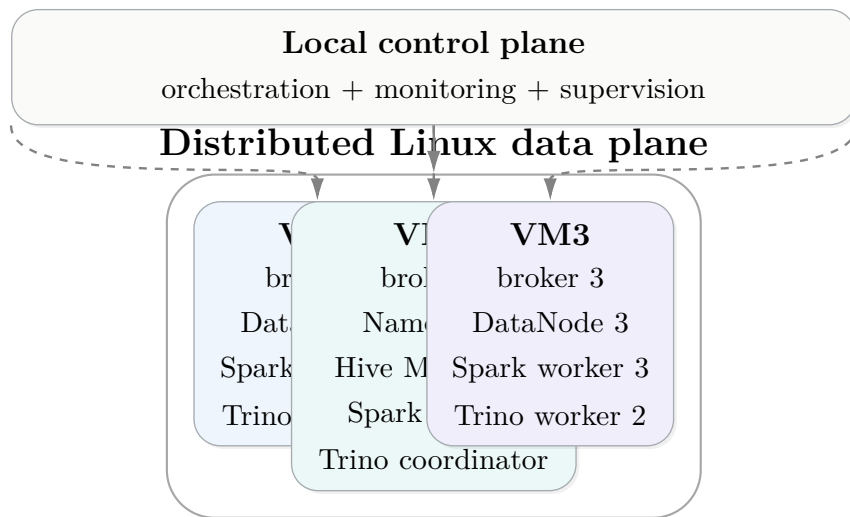
This chapter explains how AdOptimizer CDP is physically deployed and how its distributed character is demonstrated in practice. The objective is not only to say that several machines are used, but to clarify why services were separated the way they were, how data and control move across the environment, and how the final deployment remains reproducible and defensible in an academic setting. The deployment approach combines distributed-service reasoning with containerized packaging and reproducible operational startup logic [11, 7, 2].

VI.2 Control Plane Responsibilities

The project adopts a deployment strategy based on a clear separation between a **local control plane** and a **distributed data plane**. The control plane is hosted on the local workstation, while the heavy data services are distributed across the three Linux virtual machines.

This separation is important for two reasons. First, it prevents the virtual machines from being overloaded by orchestration and monitoring services that do not need to participate directly in distributed storage or processing. Second, it makes the architecture easier to explain: the control plane governs, observes, and triggers, while the data plane stores, computes, and exposes data services.

Control-plane service	Operational role	Reason for local placement
Airflow	orchestration of reset, readiness, loading, and transformation DAGs	avoids consuming VM resources for workflow control and keeps supervision centralized
Prometheus	collection of infrastructure and pipeline metrics	centralizes observability and simplifies metric management
Grafana	dashboard-based monitoring and demo evidence	provides one visual supervision point for the whole platform
Kafka UI	broker and topic inspection	keeps operational visibility outside the production data services themselves
Pipeline metrics exporter and helper runtimes	local operational support for reportable evidence and control tasks	reduces unnecessary service inflation on the Linux nodes

Table VI.1: Main responsibilities of the local control plane**Figure VI.1:** Separation between the local control plane and the distributed Linux data plane.

VI.3 VM-Level Service Separation

The data plane is not deployed as three identical generic machines. Instead, the services are deliberately separated according to their coordination role or worker role.

Node	Main hosted services	Architectural logic
VM1	Kafka broker 1, HDFS DataNode 1, Spark worker 1, Trino worker 1, cAdvisor	extends storage, compute, and query-worker capacity without carrying cluster leadership responsibilities
VM2	Kafka broker 2, Kerberos KDC, NameNode, DataNode 2, Hive Metastore, Spark master, Spark submit helper, Spark worker 2, Spark Thrift Server, Trino coordinator, HTTPS gateway, cAdvisor	acts as the leader node and integration point for coordination-heavy services
VM3	Kafka broker 3, HDFS DataNode 3, Spark worker 3, Trino worker 2, cAdvisor	reinforces the cluster with additional worker capacity and distributed persistence

Table VI.2: VM-level service allocation in the distributed deployment

This service distribution supports clearer operational reasoning. VM2 concentrates the services that require a central authority or coordination function, such as Kerberos, the HDFS namespace, the shared catalog, and the query coordinator. VM1 and VM3 remain focused on horizontal extension: they add storage blocks, Spark execution resources, Kafka quorum members, and Trino worker capacity.

From an academic perspective, this separation is stronger than a pseudo-distributed demonstration in which all important services still run on the same node. Here, the deployment logic itself becomes part of the engineering contribution.

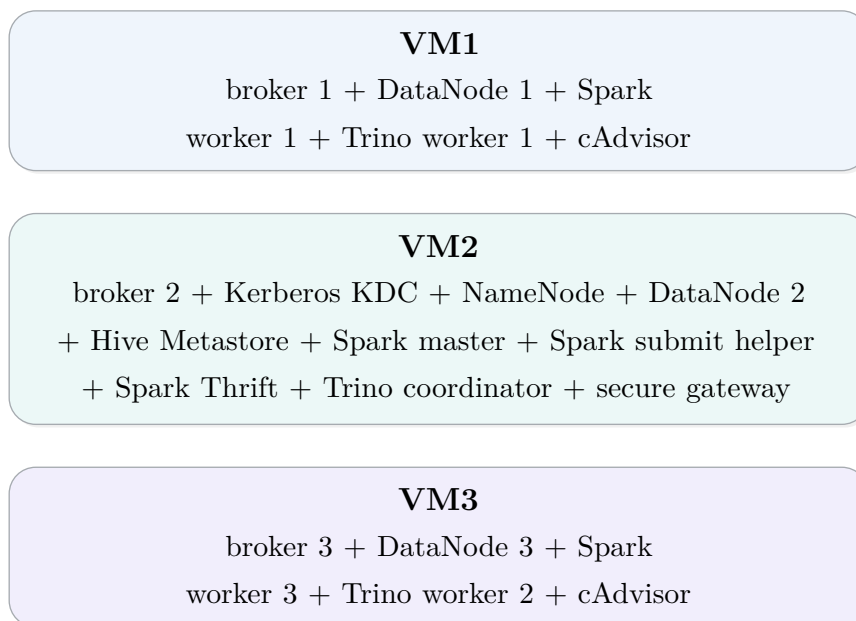


Figure VI.2: Service placement map across VM1, VM2, and VM3.

VI.4 Data Flow Across Machines

The report should clearly explain the end-to-end movement of data:

- source datasets to Kafka,
- Kafka to HDFS bronze,
- HDFS to Spark,
- Spark to Iceberg,
- Iceberg and Metastore to Trino,
- transformations and validation to final analytical exposure.

The actual machine-to-machine flow follows a layered path:

1. the control plane prepares or triggers the dataset ingestion process;
2. records are published into the three-node Kafka cluster across VM1, VM2, and VM3;
3. consumers persist the raw batches into the HDFS bronze zone governed by the NameNode on VM2 and distributed across the DataNodes on all three VMs;
4. Spark jobs are submitted remotely to VM2 and executed across the Spark workers available in the cluster;
5. Spark writes structured raw Iceberg tables and registers them through Hive Metastore on VM2;
6. Trino accesses the governed metadata and exposes SQL access through its coordinator/worker architecture;
7. dbt-spark and Great Expectations operate on top of the distributed environment from the control plane side, without collapsing the underlying data-plane separation.

This flow is important because it proves that the platform is not a single-machine ETL disguised as a distributed system. Data transport, persistence, metadata governance, processing, and analytical access cross different services and different machines with explicit operational boundaries between them.

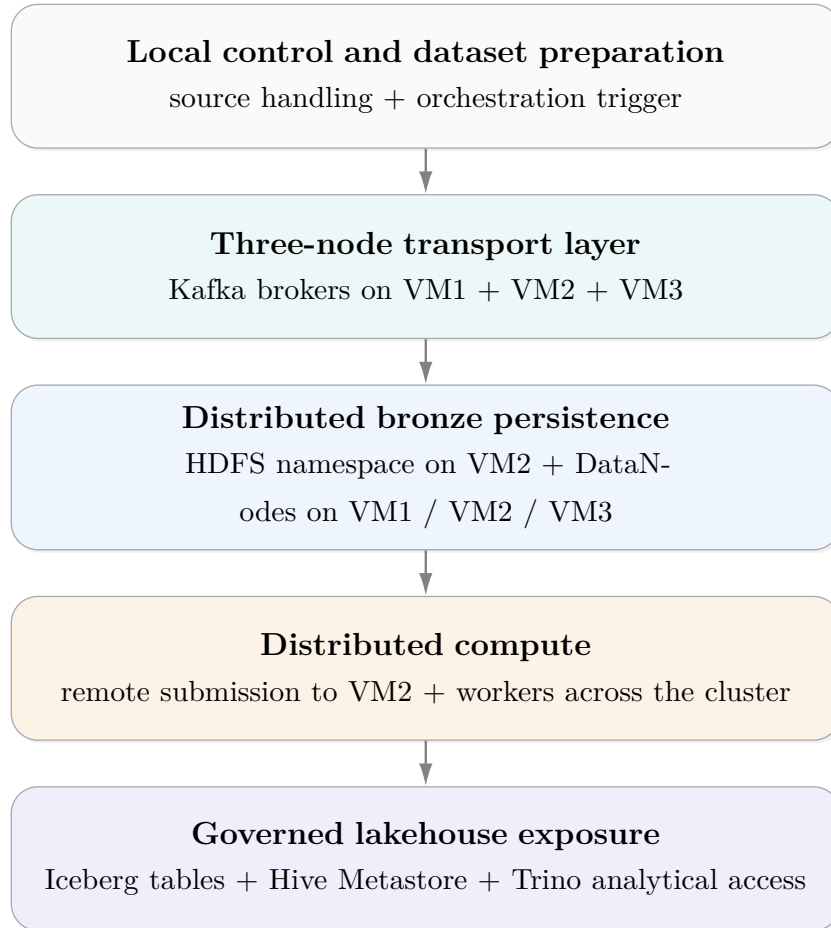


Figure VI.3: Machine-level data flow from local source handling to distributed analytical exposure.

VI.5 Distributed Processing Proof

The strongest practical proof of distributed execution in the project comes from the Spark cluster behavior. The Spark master is hosted on VM2, but executor resources are distributed across multiple workers on VM1, VM2, and VM3. This means processing capacity is not bound to a single machine.

In operational demonstrations, the Spark master interface shows several registered workers, and applications allocate executors across the available nodes depending on the execution context. This is the key difference between a truly distributed execution environment and a local-only demonstration: processing tasks can be scheduled beyond the coordinator node itself.

The deployment also includes a Spark Thrift Server and a remote Spark submission helper on VM2. These components reinforce the distributed logic of the environment and remain consistent with the Spark separation between cluster coordination and executor-side execution [7]:

- raw Spark jobs can be launched into the cluster from the control plane through a remote submission path;
- transformation-oriented SQL access can target the distributed Spark environment through the Thrift interface;
- execution is therefore coordinated centrally but materially carried out by multiple nodes.

VM3 may sometimes appear idle during lighter workloads, which is normal in a small academic cluster. Distributed execution does not require that all workers always consume identical resources at the same time. What matters is that the cluster is capable of allocating work across multiple nodes and that the platform is engineered to support that behavior.

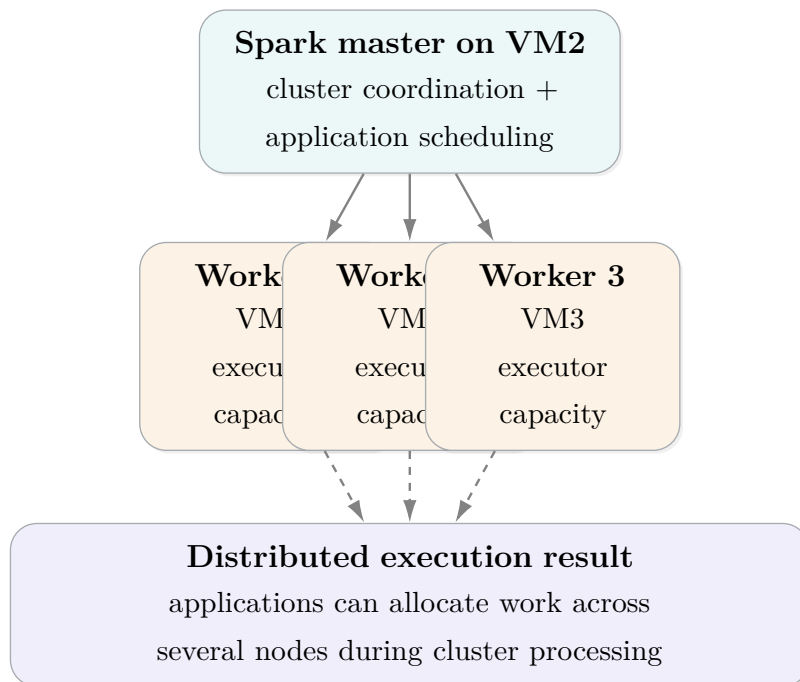


Figure VI.4: Spark master and worker collaboration across VM1, VM2, and VM3 during distributed execution.

VI.6 Operational Packaging and Deployment Bundles

The deployment is supported by explicit operational packaging rather than undocumented manual steps. Each node has its own deployment bundle, startup logic, reset logic, and preflight checks. This is a major part of the project’s reproducibility value and reflects the same infrastructure-as-documentation discipline encouraged by container-based operational practice [11].

Deployment artifact type	Role in operations	Why it matters academically
Bundle management scripts	prepare and synchronize node-specific deployment content	show that deployment is controlled and repeatable
Preflight checks	verify ports, dependencies, peer endpoints, and critical services	reduce ambiguous failure states before execution
Reset scripts	clean previous runtime state and support reproducible reruns	make demonstrations and validation cycles consistent
Startup scripts	launch services in the required sequence	formalize the operational lifecycle of the platform
Access-point documentation	record URLs, ports, and secured entry paths	improves usability and defense clarity during demonstrations

Table VI.3: Operational deployment artifacts used in the project

The startup order is also intentional: VM2 is started first because it hosts the coordination-heavy services, then VM1 and VM3 are started to extend the cluster, and finally the local control plane is brought up after the distributed data plane is stable. This sequence reflects a real dependency-aware deployment strategy rather than arbitrary service launching.

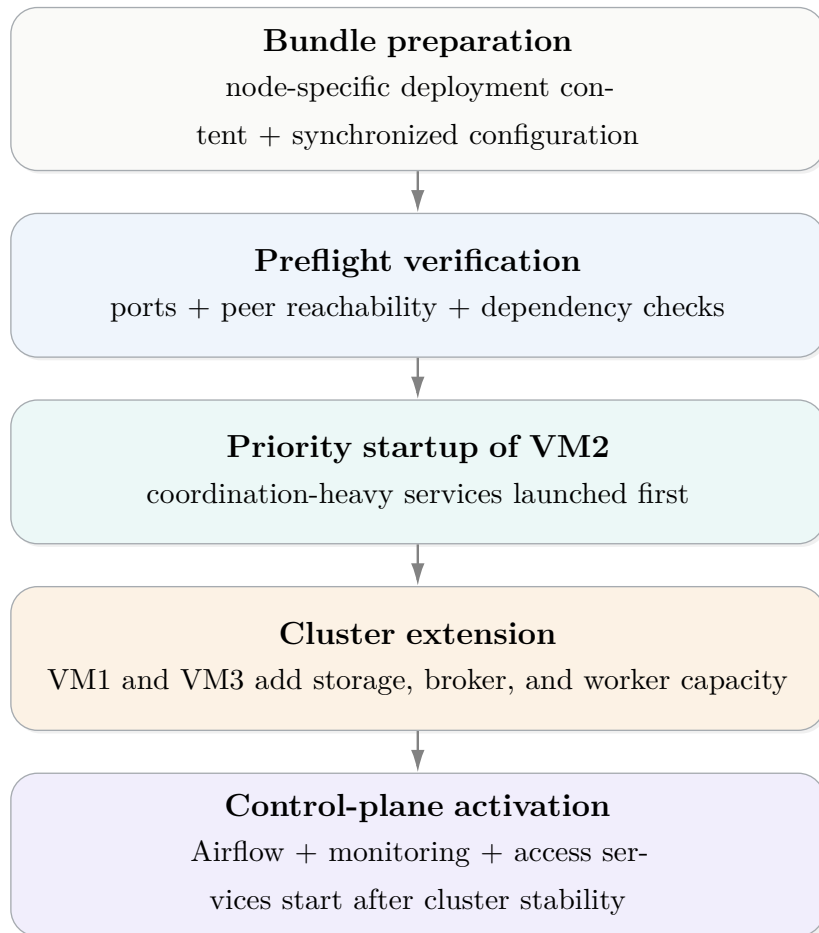


Figure VI.5: Deployment lifecycle from bundle preparation to preflight, startup order, and control-plane activation.

VI.7 Conclusion

This chapter clarified that the deployment of AdOptimizer CDP is intentionally distributed at both the service level and the machine level. The local control plane supervises the environment without collapsing it into a single node, VM2 plays the role of coordination leader, and VM1 and VM3 extend storage, compute, and worker capacity. Data movement, processing, and analytical access therefore take place across explicit multi-node boundaries.

The chapter also showed that the deployment is reproducible because it is supported by bundles, reset logic, startup scripts, and preflight checks. As a result, the distributed architecture can be defended not only as a conceptual design, but as an implemented and operational deployment strategy. The next chapter focuses on the security architecture that hardens this deployment and protects its critical access paths.

Chapter VII

Security Architecture and Hardening

VII.1 Introduction

This chapter presents the security architecture integrated into AdOptimizer CDP. The goal of the security layer is not to transform the platform into a full enterprise zero-trust environment, but to ensure that the final deployment already includes meaningful protection mechanisms for identity, browser access, sensitive service exposure, remote operations, and persisted runtime data. The implemented controls are grounded in standard secure-service principles rather than improvised protections [19, 2, 20, 9].

VII.2 Security Objectives

The security work of the project follows five main objectives:

- authenticate critical distributed services rather than relying on implicit trust;
- secure browser-facing administrative or analytical interfaces;
- protect exposed endpoints through encryption in transit;
- reduce the operational risk of remote orchestration actions;
- strengthen the protection of service data persisted on the virtual machines.

Security objective	Implemented mechanism	Protected scope
Distributed identity control	Kerberos-secured Hadoop runtime	Hadoop services and related secure interactions
Protected web access	SPNEGO + HTTPS for HDFS browser endpoints	NameNode and DataNode administrative web paths
Secured analytical exposure	authenticated Trino gateway over TLS	browser-based SQL access path
Protected orchestration path	SSH-based remote Spark submission	control-plane to VM2 operational execution
Encrypted persisted runtime data	LUKS-backed runtime storage support	service data at rest on the Linux nodes

Table VII.1: Main security objectives and corresponding implementation controls

VII.3 Kerberos-Secured Hadoop Services

Kerberos is the core identity mechanism used to secure the Hadoop side of the platform. In a distributed environment, services such as the NameNode and DataNodes should not trust requests only because they originate from another cluster component. They must validate the identity of the caller explicitly [19, 2].

In the implementation, the Kerberos KDC and administration service are hosted on VM2, which is already the coordination-heavy node of the data plane. This is consistent with the overall architecture: the node that centralizes namespace and metadata authority also centralizes identity management for the secure Hadoop runtime.

The practical contribution of Kerberos in this project is threefold:

- it gives Hadoop services authenticated identities;
- it allows secure service-to-service interactions in the distributed cluster;
- it provides the foundation for browser-based negotiated authentication through SP-NEGO.

This mechanism is especially important because it shifts the platform away from a demonstration architecture based only on open internal trust. Instead, the cluster operates with explicit authentication logic that is relevant both academically and operationally.

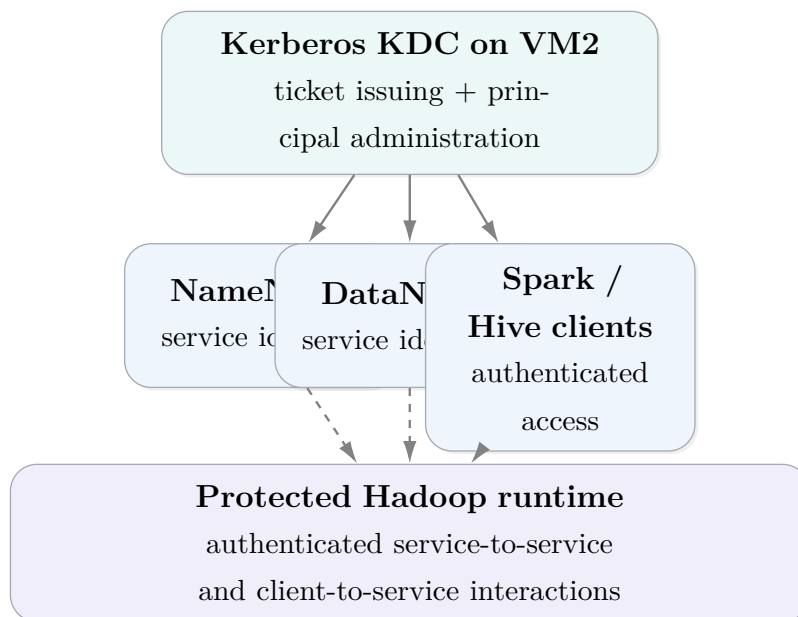


Figure VII.1: Kerberos-centered identity flow between the KDC and the protected Hadoop service layer.

VII.4 SPNEGO-Protected HDFS Browser Access

The HDFS browser path is protected through SPNEGO layered on top of the Kerberos-based identity model. This means that access to the secured NameNode and DataNode web interfaces is not treated as a public or weakly protected administrative convenience. It becomes part of the authenticated architecture [2, 19].

From a report perspective, this feature is important because administrative web interfaces are often overlooked in student projects even when the back-end services are secured. Here, the platform extends the security posture to the browser-facing HDFS access path.

The implementation required both server-side and client-side alignment:

- server-side Kerberos-aware Hadoop configuration;
- SPNEGO-protected web endpoints;
- HTTPS exposure of the HDFS browser interfaces;
- a valid Kerberos ticket on the Windows client;
- Firefox SPNEGO configuration aligned with the project hostname aliases.

This path was one of the most configuration-sensitive parts of the deployment, but once stabilized it became strong evidence that the project secured not only APIs and internal services, but also real human-facing browser access to the distributed file system.

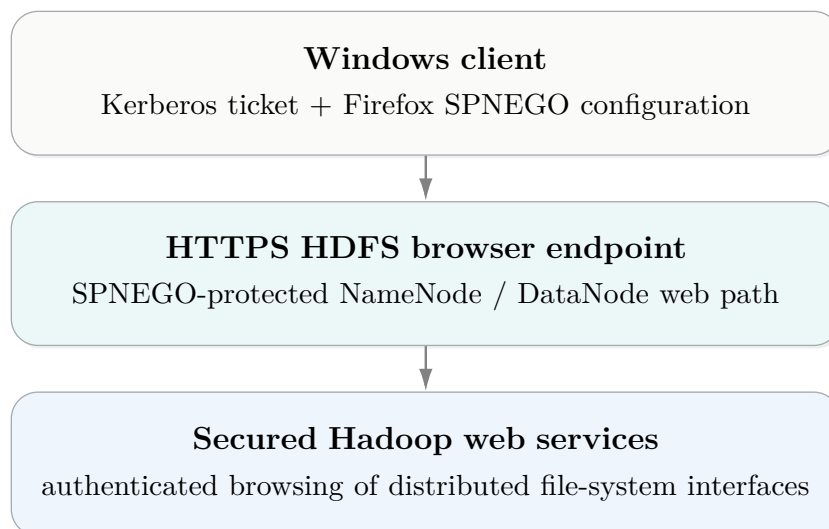


Figure VII.2: Secure HDFS browser path from Windows Kerberos ticket to SPNEGO-authenticated HTTPS endpoint.

VII.5 TLS-Protected Interfaces

TLS is used to protect the main browser-facing interfaces of the platform. The implemented objective is not to wrap every internal port blindly, but to secure the access paths that matter most for demonstration, administration, and analytical interaction. In this sense, transport protection complements identity protection rather than replacing it [2, 22].

Two main exposure zones are protected:

- the HDFS browser interfaces through HTTPS;
- the Trino analytical access gateway through HTTPS.

The local control-plane interfaces are also exposed through a local TLS gateway, which reinforces a consistent secured-access posture for Airflow, Grafana, Prometheus, and Kafka UI. This is particularly useful during demonstrations because it shows that protected service exposure is part of the operational design rather than a late-stage cosmetic enhancement.

At the same time, the project keeps an honest boundary: some trust paths remain demo-oriented, and browser warnings may still appear if the local certificate authority is not trusted on the client machine. This does not invalidate the implemented encryption layer; it simply reflects the difference between a secured academic environment and a fully managed enterprise certificate infrastructure.

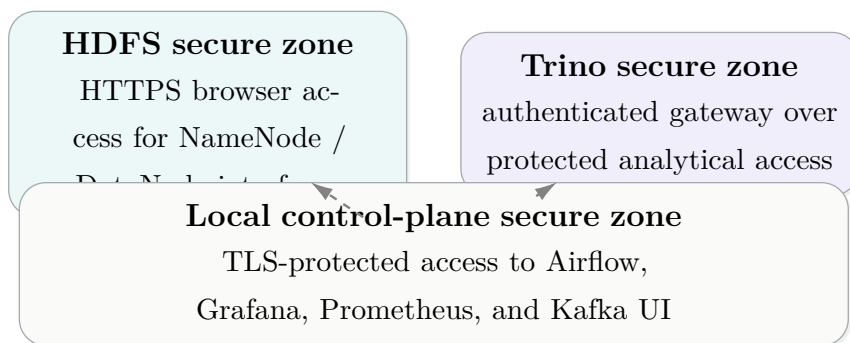


Figure VII.3: TLS-protected service exposure zones for HDFS, Trino, and the local control plane interfaces.

VII.6 Authenticated Trino Access

Trino is not exposed as an open analytical endpoint. Instead, the project implements an authenticated gateway in front of the coordinator-side browser access path. This has two important effects [22]:

- it reduces uncontrolled direct access to the analytical query service;
- it demonstrates that user-facing SQL access must also respect basic identity and access boundaries.

The gateway is hosted on VM2 together with the Trino coordinator, which is consistent with the node's role as the entry point for coordination-heavy services. Through this design, the browser-facing access path is separated from the internal worker topology. Users connect to the protected entry point, while the internal distributed Trino execution still takes place through the coordinator and worker nodes.

This is a meaningful security improvement for the report because it shows that data exposure was considered at the query interface level, not only at the storage or transport level.

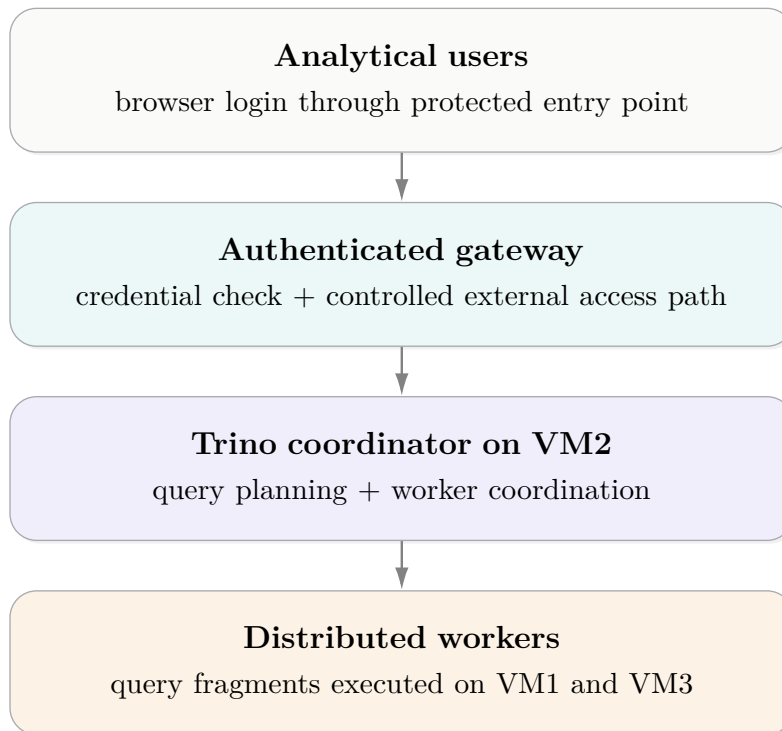


Figure VII.4: Authenticated Trino gateway in front of coordinator and worker execution.

VII.7 SSH-Based Remote Spark Submission

The orchestration model of the project relies on a remote Spark submission path from the control plane to VM2. This was introduced to avoid unstable local-driver assumptions and to keep the operational launch path aligned with the distributed cluster environment.

From a security perspective, this means the orchestration node must not submit jobs through uncontrolled or manually repeated remote actions. The platform therefore uses an SSH-based remote execution path with dedicated key material and documented access preparation [20].

This design improves the platform in three ways:

- it keeps the control plane separate from the heavy Spark runtime;
- it avoids exposing the orchestration flow through weaker manual execution habits;
- it provides a consistent operational bridge between local Airflow logic and the distributed cluster leader on VM2.

In the broader security architecture, this remote path is not only an operational convenience. It is the secured mechanism through which orchestration reaches distributed compute.

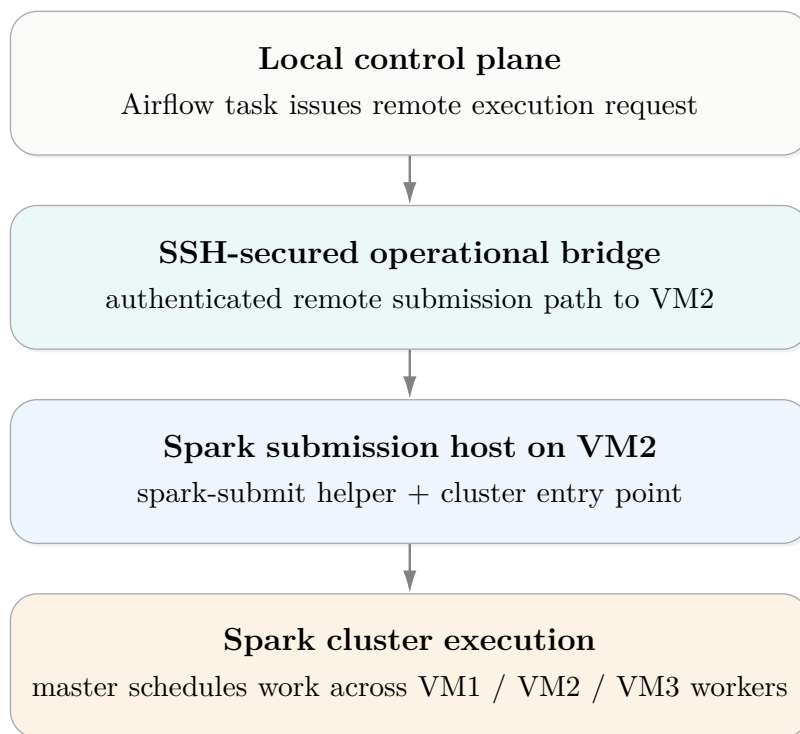


Figure VII.5: Secured control-plane-to-VM2 remote Spark submission path over SSH.

VII.8 Encrypted Runtime Storage Support

The project also introduces support for encrypted runtime storage through LUKS on the Linux virtual machines. This measure targets **data at rest** rather than only service authentication or traffic encryption [9].

In practice, this control is intended to protect runtime service data stored on VM1, VM2, and VM3. The main idea is that a distributed platform should not protect data movement while leaving persisted service data entirely unprotected. Even in an academic setup, demonstrating this distinction is valuable because it shows a more complete understanding of security dimensions.

The implemented logic therefore complements the other controls:

- Kerberos protects identity,
- SPNEGO protects browser negotiation,
- TLS protects exposed traffic,
- LUKS protects persisted runtime storage.

This layered view helps the report explain that security was approached as a set of complementary boundaries rather than a single technology checkbox.

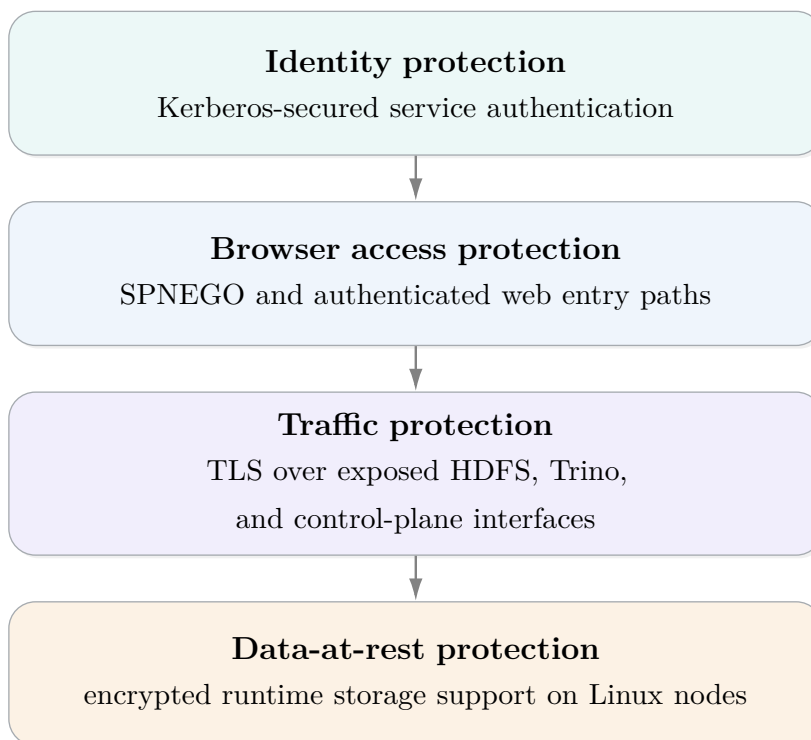


Figure VII.6: Security coverage map distinguishing identity, browser access, traffic encryption, and encrypted runtime storage.

VII.9 Security Boundaries and Remaining Limits

The implemented security layer is meaningful, but it must be described honestly. The project demonstrates baseline infrastructure hardening suitable for an academic distributed platform; it does not claim to be a finished enterprise security program.

The current implementation can validly claim:

- Kerberos-secured Hadoop runtime;
- SPNEGO-protected HDFS browser access;
- TLS-protected browser-facing endpoints;
- authenticated Trino access;
- SSH-based remote Spark submission;
- encrypted runtime storage support on the Linux nodes.

At the same time, some limits remain outside the strict PFE scope:

- enterprise-scale PKI and certificate lifecycle automation;

- full centralized secret management;
- fine-grained authorization frameworks across all analytical consumers;
- large-scale security auditing, SIEM integration, or intrusion detection;
- organization-wide identity federation.

This distinction is not a weakness in the report. On the contrary, it strengthens its credibility by showing a mature understanding of what was actually implemented and what remains future hardening work.

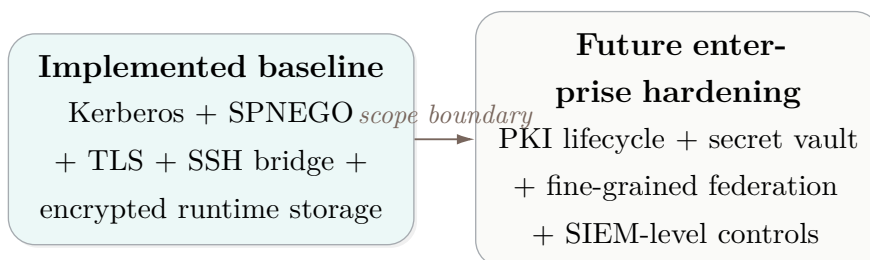


Figure VII.7: Implemented security boundary versus out-of-scope future enterprise hardening controls.

VII.10 Conclusion

This chapter showed that security is integrated into AdOptimizer CDP as an architectural layer rather than a superficial add-on. Identity is controlled through Kerberos, browser-based Hadoop access is protected through SPNEGO and HTTPS, analytical exposure is filtered through an authenticated Trino gateway, remote Spark submission is secured operationally through SSH, and encrypted runtime storage strengthens protection at rest.

The chapter also made clear that the implemented security posture is honest in scope: it is strong enough to be technically meaningful and academically defensible, while still distinct from a full enterprise security stack. The next chapter focuses on how this distributed and secured platform was validated through readiness checks, pipeline execution, observability, and demonstration evidence.

Chapter VIII

Validation, Testing, and Results

VIII.1 Introduction

This chapter presents the validation methodology and the main technical results of AdOptimizer CDP. The objective of the validation work is to prove that the platform is not only well designed, but also operational, distributed, monitored, and secured in practice. The chapter therefore focuses on evidence-oriented validation rather than on theoretical expectation alone. In that sense, the validation logic remains aligned with the broader engineering principles already discussed in the report: distributed services must be verifiable through observable runtime evidence, not only through architectural intent [15, 1, 21]. To preserve a clean results-oriented structure, the chapter is intentionally organized around textual interpretation, summary tables, and operational screenshots discussed as evidence rather than around conceptual diagrams.

VIII.2 Validation Strategy

The validation strategy was structured around several complementary dimensions:

- infrastructure readiness;
- end-to-end orchestration readiness;
- distributed execution evidence;
- secure-access verification;
- data-quality and observability evidence.

Validation axis	What was checked	Expected proof
Infrastructure validation	service endpoints, cluster health, secured ports, cluster membership	the platform components are reachable and stable
Pipeline validation	Airflow DAG execution path and task sequencing	the operational workflow can run in the intended order
Distributed execution validation	multi-worker participation and cross-node service roles	the architecture is truly distributed, not pseudo-distributed
Security validation	Kerberos, SPNEGO, TLS, authenticated entry points	protected access paths are functioning as designed
Observability validation	dashboards, metrics, exporters, and run history	the platform exposes usable operational evidence

Table VIII.1: Validation strategy adopted for the deployed platform

This strategy is particularly important for the jury because it transforms the project defense into an evidence-based argument. Instead of claiming that the platform works only in principle, the report can point to concrete service checks, DAG outcomes, monitoring views, and secured access demonstrations. This way of validating a distributed platform is consistent with the idea that reliable systems must be defended through explicit operational signals rather than by static design claims alone [15, 21].

VIII.3 Infrastructure Validation

Infrastructure validation focused on proving that the main services were correctly deployed and reachable in the final environment. This includes the Kafka cluster, HDFS cluster, Hive Metastore, Spark cluster, Trino cluster, Kerberos services, browser-facing TLS gateways, and the monitoring stack [5, 2, 3, 7, 22, 19, 21, 13].

At the service level, the validation campaign confirmed the following key conditions:

- Kafka brokers were reachable across VM1, VM2, and VM3;
- HDFS reported an active NameNode and live DataNodes across the three VMs;
- Hive Metastore was reachable and usable as the metadata authority;
- Spark master, workers, history server, and Thrift access were available;
- Trino coordinator and workers were aligned with the secured access path;
- Kerberos and related secure endpoints were reachable;
- Prometheus, Grafana, and exporter services were available from the control plane.

This infrastructure validation is essential because it establishes the platform baseline before any pipeline result can be interpreted. A successful data run is only meaningful if the underlying cluster services are themselves healthy and correctly interconnected.

The corresponding proof for this section is therefore presented through service-state tables, browser checks, and runtime screenshots rather than through a conceptual schema. This choice is more consistent with the role of Chapter VIII, which is to show observed results from the implemented platform.

VIII.4 Pipeline Validation

Pipeline validation was centered on the five operational DAGs that structure the lifecycle of the platform:

1. environment reset pipeline;
2. lakehouse setup pipeline;
3. lakehouse readiness pipeline;
4. Kafka-HDFS-Spark lakehouse pipeline;
5. dbt-spark lakehouse pipeline.

The role of this validation was not merely to confirm that Airflow can trigger tasks. It was to prove that the platform can execute a controlled workflow with dependency-aware sequencing and observable task outcomes [1]:

- return to a clean baseline;
- reinitialize its technical structures;
- verify readiness before deeper execution;
- materialize the raw lakehouse foundation;
- and continue toward transformation-oriented analytical preparation.

The environment reset and lakehouse readiness phases are especially important because they transform troubleshooting experience into formal operational logic. Rather than depending on ad hoc manual cleanup, the platform can be defended as having explicit lifecycle control.

In the final presentation of results, this section is therefore better supported by Airflow run screenshots, task-state tables, and execution summaries than by conceptual flow diagrams.

DAG	Validation meaning	Evidence type
Environment reset	proves reproducible cleanup and reset capability	Airflow task history, logs, and end-state verification
Lakehouse setup	proves technical structures can be prepared consistently	service-side readiness after initialization
Lakehouse readiness	proves the platform checks its own critical services before deeper processing	positive readiness tasks across HDFS, Hive, Spark, and Trino
Kafka-HDFS-Spark pipeline	proves raw distributed ingestion and lakehouse loading path	topic activity, bronze persistence, Spark execution, raw lakehouse outputs
dbt-spark pipeline	proves modeled transformation logic can operate on the distributed lakehouse foundation	model execution traces, curated outputs, validation coupling

Table VIII.2: Operational meaning of the core DAG validations

VIII.5 Distributed Execution Validation

Distributed execution validation aimed to prove that processing and service behavior are not confined to one node. The evidence comes from several complementary observations:

- Spark master visibility over workers located on multiple VMs;
- HDFS DataNode participation across VM1, VM2, and VM3;
- Kafka broker distribution across the three-node cluster;
- Trino coordinator/worker separation across the VMs.

The most direct proof concerns Spark. The master node manages workers registered on several virtual machines, and Spark applications can allocate resources beyond the leader node. Even when light workloads do not saturate every worker equally, the cluster topology and execution path still demonstrate a true distributed compute model [7].

Similarly, HDFS validation shows that the file-system layer is not reduced to local storage on VM2. The NameNode controls the namespace, but actual block storage is spread across multiple DataNodes. This is a critical argument for the defense because it proves that the storage layer also participates in the distributed design [2].

This part of the chapter is intended to be documented through runtime screenshots and compact comparison tables showing node participation, worker registration, and distributed service presence across the VMs.

VIII.6 Security Validation

Security validation focused on proving that the implemented protection mechanisms are not only present in configuration files, but usable in practice [19, 2, 22, 20, 9].

The main validated points are:

- Kerberos-secured Hadoop services were active in the deployment;
- the HDFS browser path functioned through the Windows Kerberos + Firefox SPNEGO configuration once the client path was correctly aligned;
- HTTPS-protected HDFS web access was operational;
- the Trino gateway exposed a protected analytical access path;
- the control plane interfaces were exposed through secured local entry points.

One especially important result is the final browser-based HDFS validation. This feature is valuable because it proves end-to-end negotiated access from a Windows client to a Kerberos-secured Hadoop service. At the same time, the report must remain honest: this path is sensitive to exact client-side configuration and is therefore presented as validated but configuration-dependent, not as universally frictionless.

The evidence retained for this section is consequently expected to come from secured browser access screenshots, authenticated interface checks, and concise result tables summarizing the validated protection paths.

VIII.7 Observability and Monitoring Validation

Observability validation confirms that the platform does not rely only on raw container logs for diagnosis. Instead, it exposes a structured monitoring layer composed of:

- Prometheus for metric collection;
- Grafana for dashboard-based visibility;
- cAdvisor on VM1, VM2, and VM3 for node/container metrics;
- pipeline-related exporters and Airflow run history for workflow evidence.

This monitoring layer matters academically because it strengthens the maturity of the platform. A distributed system is harder to defend when failures can only be explained

retrospectively through manual shell inspection. Here, the project shows that service states, cluster readiness, and pipeline outcomes are visible through explicit observability mechanisms [21, 13, 1].

The existence of dashboards, exporters, and centralized metric collection also contributes to the evidence strategy of the report. It allows the platform to be presented not only as running, but as observable and diagnosable.

For the same reason, the observability discussion in this chapter is best documented through dashboard captures and metric-oriented summary tables rather than through additional conceptual illustrations.

VIII.8 Main Results

The main technical result of the project is the delivery of a distributed, orchestrated, monitored, and security-aware customer-data lakehouse platform under the AdOptimizer CDP scope.

The final validated outcome can be summarized as follows:

- heterogeneous customer-related datasets can be ingested through a controlled Kafka path;
- raw data can be persisted in a replayable bronze layer on distributed HDFS;
- Spark can structure and load raw lakehouse tables in a distributed environment;
- Hive Metastore centralizes metadata for governed table access;
- Trino provides analytical SQL exposure over the lakehouse foundation;
- dbt-spark introduces modeled transformation discipline;
- Great Expectations provides quality-oriented evidence;
- Airflow provides explicit lifecycle orchestration;
- Prometheus and Grafana provide operational visibility;
- Kerberos, SPNEGO, TLS, authenticated gateways, and encrypted runtime storage strengthen the final deployment.

The most important interpretive result is that the platform can be defended as a **Customer 360-ready analytical foundation**, not as a finished enterprise AI product. This distinction is central to the academic honesty of the report.

VIII.9 Conclusion

This chapter demonstrated that AdOptimizer CDP was validated operationally and not only designed conceptually. Service readiness, pipeline execution, distributed participation, secure access paths, and observability mechanisms were all used to build an evidence-based understanding of the final platform.

As a result, the project can be defended as a deployed and tested distributed data engineering system that provides a trustworthy base for later Customer 360 and AI-oriented work. The next chapter takes a more critical perspective by discussing the strengths, trade-offs, academic value, industrial relevance, and remaining limits of the final architecture.

Chapter IX

Discussion and Critical Analysis

IX.1 Introduction

This chapter provides a critical interpretation of the implemented work. Its purpose is to move beyond pure description and explain what the project achieves well, where its trade-offs appear, what it contributes academically and industrially, and which boundaries must remain explicit in a serious jury defense. The discussion therefore reconnects the final implementation with the reference ideas of distributed data-intensive systems and modern lakehouse-oriented engineering [15, 8].

IX.2 Strengths of the Final Architecture

The final architecture presents several important strengths. Many of them correspond directly to the architectural qualities discussed in the literature on distributed data-intensive systems and on lakehouse-oriented analytics platforms [15, 8].

First, it is **modular**. Ingestion, bronze storage, processing, metadata governance, SQL exposure, transformation, validation, orchestration, monitoring, and security are clearly separated. This makes the platform easier to reason about and strengthens the quality of the report because each layer has a clear responsibility.

Second, it is **truly distributed**. Kafka, HDFS, Spark, and Trino are all deployed across more than one virtual machine, and the architecture does not depend on a single overloaded “fake cluster” node for every important function [5, 2, 7, 22].

Third, it is **governance-oriented**. The project does not stop at moving data. It introduces shared metadata, structured modeling, data-quality validation, orchestration logic, and observability. This is a major strength because it positions the work as a serious data

engineering platform rather than a simple ingestion pipeline [3, 10, 14, 21].

Fourth, it includes **meaningful security integration**. The security controls are not limited to a conceptual recommendation. Kerberos, SPNEGO, TLS, authenticated gateways, encrypted runtime storage support, and secured remote orchestration were actually integrated into the deployment logic [19, 2, 20, 9].

For consistency with the intended style of the discussion chapter, these strengths are expressed only through analytical prose and summary tables, without adding interpretive diagrams.

IX.3 Trade-Offs and Constraints

The architecture also reflects real constraints and trade-offs.

The first major constraint is the **limited infrastructure capacity** of the academic environment. The cluster is distributed, but it is built on lightweight virtual machines with limited memory, storage, and compute capacity. This makes the deployment realistic enough for distributed proof, but it also imposes stricter discipline in dataset sizing, runtime cleanup, and service tuning.

The second trade-off is **integration complexity**. A platform combining Kafka, HDFS, Spark, Hive Metastore, Trino, dbt-spark, Great Expectations, Airflow, and multiple security controls is inherently more complex than a monolithic pipeline. That complexity is justified by the learning and platform value, but it also increases debugging effort and operational sensitivity [15].

The third trade-off is **security overhead**. Once Kerberos, SPNEGO, TLS, and encrypted runtime storage are introduced, the project becomes much stronger architecturally, but also harder to configure, restart, and validate. This is not a flaw of the work; it is the natural consequence of moving from an open demo cluster to a more realistic protected environment.

Constraint	Observed impact	Architectural response
Limited VM resources	tighter storage and memory pressure	reduced dataset scope and disciplined service separation
Multi-tool integration complexity	longer debugging and stabilization cycles	explicit DAG boundaries, preflight checks, and bundle-based deployment
Security hardening overhead	more fragile configuration paths for some services	documented access paths and layered security design

Table IX.1: Main constraints and the corresponding design responses

IX.4 Academic Value

The project has strong academic value because it combines several disciplines that are often studied separately:

- distributed systems;
- Big Data storage and processing;
- lakehouse architecture;
- data quality and governance;
- workflow orchestration;
- observability;
- and infrastructure security.

This interdisciplinary nature makes the project more than a stack assembly exercise. It demonstrates how theoretical principles such as separation of concerns, distributed coordination, schema management, replayability, and secure service exposure can be translated into an operational platform [15, 8, 18].

It also provides academic honesty in scope. The report does not confuse foundational customer-data engineering with the final AI layer. Instead, it positions the work as the technical basis that future segmentation, churn, value-prediction, or personalization modules could consume. This is a stronger academic position than claiming prematurely that the AI product itself is already complete.

IX.5 Industrial Value

The industrial value of the platform lies in its role within the broader AdOptimizer AI initiative. SMART AUTOMATION TECHNOLOGIES needs a structured and trustworthy customer-data foundation before downstream intelligent marketing services can operate reliably.

In that context, AdOptimizer CDP contributes industrially by:

- centralizing heterogeneous customer-related data in a governed architecture;
- creating a repeatable path from source datasets to analytical data products;
- exposing SQL-accessible and transformation-ready outputs;

- preserving observability and security as operational requirements;
- preparing a realistic base for future Customer 360 and AI-driven services.

This means the project is not an isolated academic prototype, but an engineering foundation aligned with a larger company vision.

IX.6 Critical Limits

The final report must still distinguish clearly between implemented work and downstream possibilities.

The project **does implement**:

- distributed ingestion, storage, processing, querying, and transformation foundations;
- metadata governance and quality validation;
- orchestration and observability;
- meaningful baseline infrastructure security.

The project **does not claim to implement fully**:

- a complete enterprise Customer 360 master data system;
- production-scale machine learning pipelines;
- real-time personalization engines;
- advanced recommendation systems;
- organization-wide security governance and policy automation.

This distinction is essential. It protects the credibility of the report and allows the jury to see the project as both ambitious and disciplined. The real achievement is the platform foundation, not an exaggerated promise that every downstream intelligent capability is already delivered.

IX.7 Conclusion

This chapter showed that the final architecture combines significant strengths with real and understandable constraints. The project is modular, distributed, governance-oriented,

monitored, and security-aware, which gives it strong academic and industrial relevance. At the same time, limited infrastructure, integration complexity, and secure-cluster operational overhead impose real trade-offs that had to be managed carefully.

Most importantly, the chapter clarified the correct positioning of the work: AdOptimizer CDP is a delivered distributed customer-data platform under the broader AdOptimizer AI vision, not the final AI product itself. The last chapter now concludes the report and outlines the most relevant future perspectives that can build on this foundation.

Chapter X

General Conclusion and Perspectives

X.1 General Conclusion

AdOptimizer CDP delivers a distributed and secured customer-data lakehouse platform implemented within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The project demonstrates that heterogeneous customer-oriented data can be ingested, transported, persisted, processed, transformed, validated, exposed, monitored, and protected through a coherent multi-node architecture. In academic terms, the work proves the feasibility of a modern Big Data Engineering platform that combines distributed systems principles, lakehouse logic, orchestration, governance, observability, and baseline infrastructure security in one integrated implementation [15, 8].

X.2 Main Achievements

The project establishes a practical foundation that combines:

- distributed ingestion through Kafka,
- distributed bronze persistence through HDFS,
- distributed processing through Spark,
- governed metadata and lakehouse organization through Hive Metastore and Iceberg,
- analytical SQL exposure through Trino,
- transformation discipline through dbt-spark,
- data-quality validation through Great Expectations,

- explicit orchestration through Airflow,
- centralized observability through Prometheus and Grafana,
- and integrated security controls through Kerberos, SPNEGO, TLS, authenticated gateways, SSH-based remote execution, and encrypted runtime storage support.

These achievements are important because they collectively position the project as a genuine platform contribution rather than a narrow pipeline demonstration. They show the practical convergence of the customer-data engineering concerns discussed throughout the report: durable raw history, governed analytical layers, distributed execution, and controlled operational exposure [15, 8, 18].

X.3 Future Perspectives

The platform can later support:

- deeper Customer 360 consolidation across additional customer sources,
- richer curated marts and semantic business layers,
- customer segmentation and persona generation,
- churn prediction and early-warning models,
- lifetime value estimation,
- personalization and campaign optimization services,
- recommendation and decision-support modules under the broader AdOptimizer AI initiative.

From an engineering perspective, future work can also strengthen the platform itself by extending fine-grained access control, improving certificate and secret lifecycle management, expanding evidence automation, and scaling the deployment toward larger workloads.

Bibliography

- [1] Apache Software Foundation. Apache airflow documentation, 2026. Accessed for technical reference.
- [2] Apache Software Foundation. Apache hadoop documentation, 2026. Accessed for technical reference.
- [3] Apache Software Foundation. Apache hive documentation, 2026. Accessed for technical reference.
- [4] Apache Software Foundation. Apache iceberg documentation, 2026. Accessed for technical reference.
- [5] Apache Software Foundation. Apache kafka documentation, 2026. Accessed for technical reference.
- [6] Apache Software Foundation. Apache parquet documentation, 2026. Accessed for technical reference.
- [7] Apache Software Foundation. Apache spark documentation, 2026. Accessed for technical reference.
- [8] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [9] cryptsetup Project. cryptsetup and luks documentation, 2026. Accessed for technical reference.
- [10] dbt Labs. dbt developer hub documentation, 2026. Accessed for technical reference.
- [11] Docker, Inc. Docker documentation, 2026. Accessed for technical reference.
- [12] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.
- [13] Grafana Labs. Grafana documentation, 2026. Accessed for technical reference.

- [14] Great Expectations. Great expectations documentation, 2026. Accessed for technical reference.
- [15] Martin Kleppmann and Chris Riccomini. *Designing Data-Intensive Applications*. O'Reilly Media, Inc., 2 edition, 2026. 2nd edition.
- [16] Jay Kreps. Questioning the lambda architecture. <https://www.confluent.io/blog/questioning-the-lambda-architecture/>, 2014. Accessed on August 20, 2026.
- [17] Doug Laney. 3d data management: Controlling data volume, velocity, and variety. Technical report, META Group, 2001.
- [18] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.
- [19] Massachusetts Institute of Technology. Mit kerberos documentation, 2026. Accessed for technical reference.
- [20] OpenSSH Project. Openssh documentation, 2026. Accessed for technical reference.
- [21] Prometheus Authors. Prometheus documentation, 2026. Accessed for technical reference.
- [22] Trino Software Foundation. Trino documentation, 2026. Accessed for technical reference.

Appendix A

Appendices

Appendix Organization

This appendix section groups the supporting material that complements the main report without interrupting the flow of the core chapters. It is intended to host operational evidence, configuration extracts, deployment references, and selected validation artifacts that substantiate the implemented platform.

A.1 Deployment and Access Evidence

This appendix can host the practical material used to deploy and operate the final environment:

- deployment commands,
- service access matrices,
- startup and reset order references,
- and selected access-point summaries for the control plane and the three Linux virtual machines.

A.2 Configuration and Runtime Extracts

This appendix can gather short configuration excerpts that are too detailed for the body chapters but useful for technical verification:

- environment variables,

- service configuration extracts,
- DAG-related runtime parameters,
- and selected catalog or cluster settings.

A.3 Validation and Screenshot Evidence

This appendix can centralize the final evidence set used to support the validation chapter:

- screenshots of HDFS, Spark, Trino, Airflow, and monitoring interfaces,
- DAG execution evidence,
- selected data-quality outputs,
- and representative observability views.

A.4 Evidence Location in the Report Workspace

The corresponding raw support material is organized primarily inside:

- 90_assets/
- 91_evidence/
- 99_submission_package/